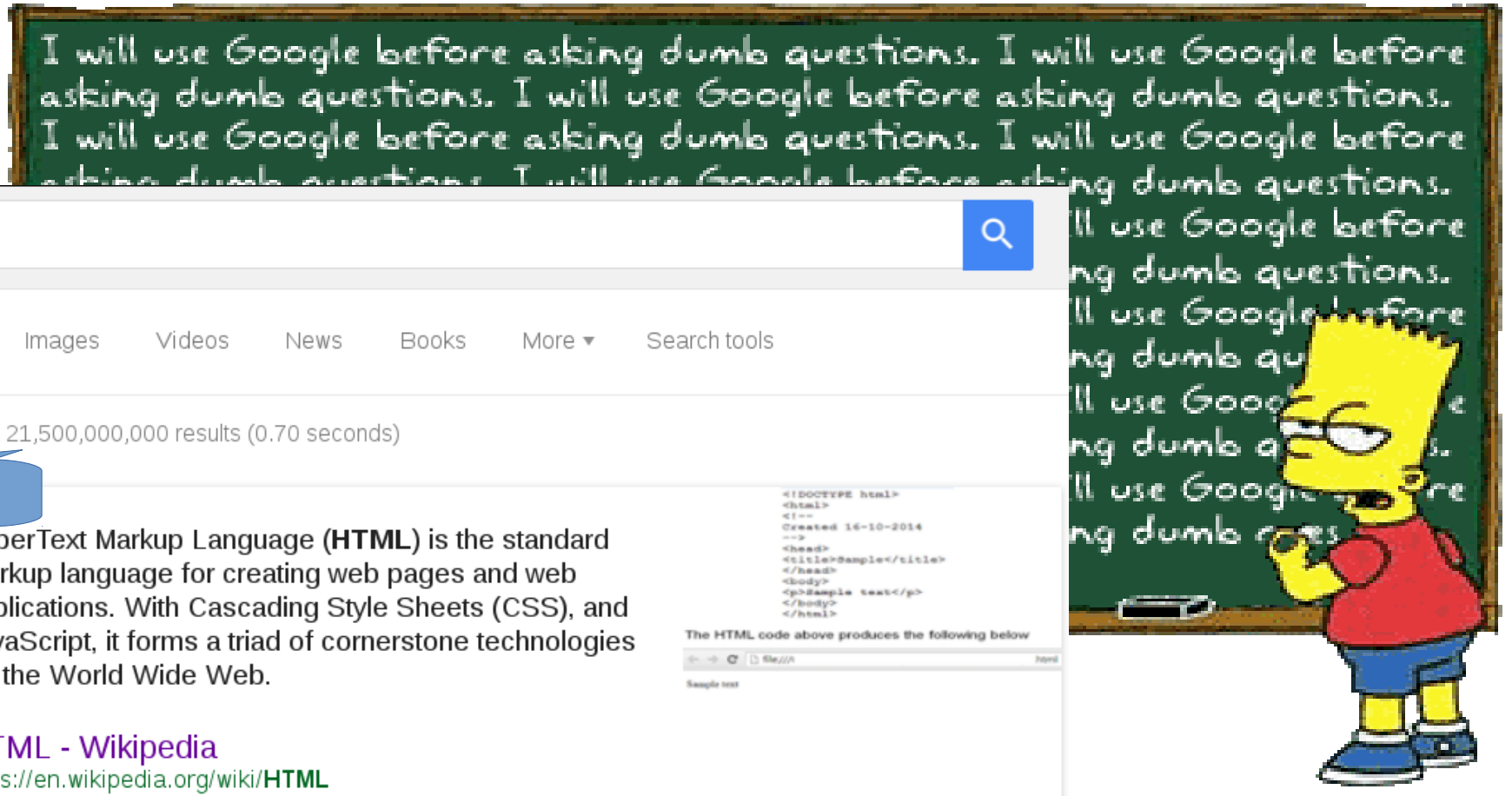
arnaud.nauwynck@gmail.com

Introduction to HTML / JavaScript / DOM

This document:
http://arnaud-nauwynck.github.io/docs/Intro-Html-JS-DOM.pdf

Agenda

- Introduction to Client-Side Web Technologies
  - HTML = Elements / Custom Elements
  - CSS
  - JavaScript
- The DOM … how JavaScript interact
- JavaScript framework(s)
  - AngularJS, binding principles



Google

html

All Images Videos News Books More Search tools

About 21,500,000,000 results (0.70 seconds)

21 Billions...

HyperText Markup Language (**HTML**) is the standard markup language for creating web pages and web applications. With Cascading Style Sheets (CSS), and JavaScript, it forms a triad of cornerstone technologies for the World Wide Web.

HTML - Wikipedia
https://en.wikipedia.org/wiki/**HTML**

About this result • Feedback

People also ask

What is HTML language?

When HTML was invented?

What is the most recent version of HTML?

Rank#1 : w3scools (not Wikipedia?)

HTML Tutorial - W3Schools
www.w3schools.com/**html**/

The **HTML** Certificate documents your knowledge of **HTML**. The CSS Certificate documents your

HTML5 Tutorial

HTML HOME
HTML Introduction
HTML Editors
HTML Basic
HTML Elements
HTML Attributes
HTML Headings
HTML Paragraphs
HTML Styles
HTML Formatting
HTML Quotations
HTML Computercode
HTML Comments
HTML Colors
HTML CSS
HTML Links
HTML Images
HTML Tables
HTML Lists
HTML Blocks
HTML Classes
HTML Iframes
HTML JavaScript
HTML Head
HTML Layout
HTML Responsive
HTML Entities
HTML Symbols
HTML Charset

HTML Introduction

❮ Previous

Next ❯

What is HTML?

HTML is the standard markup language for creating Web pages.

- HTML stands for Hyper Text Markup Language
- HTML describes the structure of Web pages using markup
- HTML elements are the building blocks of HTML pages
- HTML elements are represented by tags
- HTML tags label pieces of content such as "heading", "paragraph", "table", and so on
- Browsers do not display the HTML tags, but use them to render the content of the page

A Simple HTML Document

Example

```
<!DOCTYPE html>
<html>
<head>
<title>Page Title</title>
</head>
```

Templates by W3.CSS

COLOR PICKER

HyperText Markup Langage

WIKIPEDIA
The Free Encyclopedia

Main page
Contents
Featured content
Current events
Random article
Donate to Wikipedia
Wikipedia store

Interaction
Help
About Wikipedia
Community portal
Recent changes
Contact page

Tools
What links here
Related changes
Upload file
Special pages
Permanent link
Page information
Wikidata item
Cite this page

Print/export
Create a book
Download as PDF
Printable version

HTML

From Wikipedia, the free encyclopedia
(Redirected from Html)

".htm" and ".html" redirect here. For other uses, see HTM (disambiguation).
For the use of HTML on Wikipedia, see Help:HTML in wikitext.

HyperText Markup Language (**HTML**) is the standard markup language for creating web pages and web applications. With Cascading Style Sheets (CSS), and JavaScript, it forms a triad of cornerstone technologies for the World Wide Web.[1] Web browsers receive HTML documents from a webserver or from local storage and render them into multimedia web pages. HTML describes the structure of a web page semantically and originally included cues for the appearance of the document.

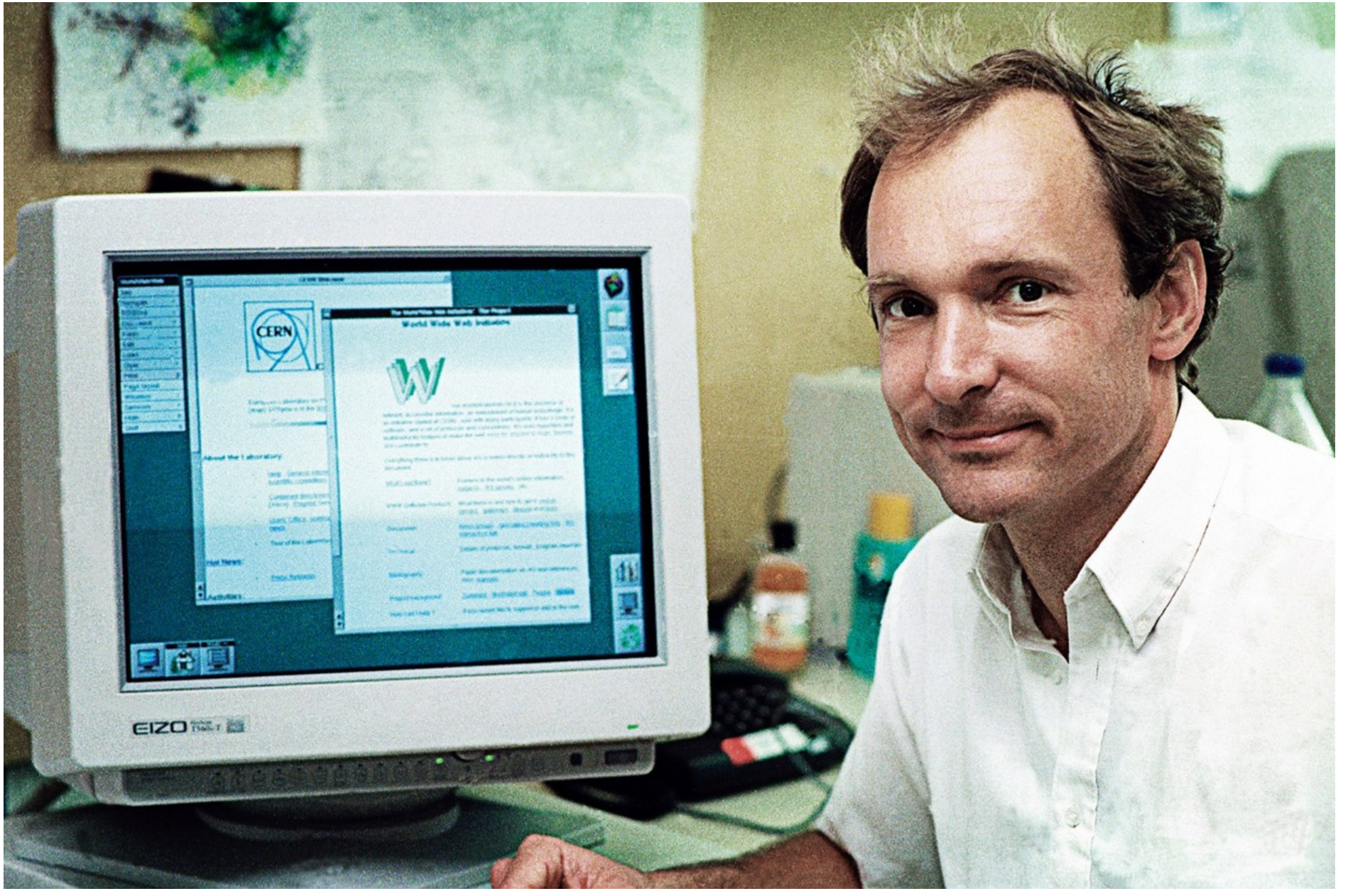
HTML elements are the building blocks of HTML pages. With HTML constructs, images and other objects, such as interactive forms may be embedded into the rendered page. It provides a means to create structured documents by denoting structural semantics for text such as headings, paragraphs, lists, links, quotes and other items. HTML elements are delineated by *tags*, written using angle brackets. Tags such as `<img />` and `<input />` introduce content into the page directly. Others such as `<p>...</p>` surround and provide information about document text and may include other tags as sub-elements. Browsers do not display the HTML tags, but use them to interpret the content of the page.

HTML can embed programs written in a scripting language such as JavaScript which affect the behavior and content of web pages. Inclusion of CSS defines the look and layout of content. The World Wide Web Consortium (W3C), maintainer of both the HTML and the CSS standards, has encouraged the use of CSS over

HTML
(HyperText Markup Language)

Filename extension	`.html` `.htm`
Internet media type	`text/html`
Type code	TEXT
Developed by	W3C & WHATWG
Initial release	1993; 23 years ago
Latest release	5.0 / 5.1 (working

1989 : HTTP + HTML
Tim Berners-Lee



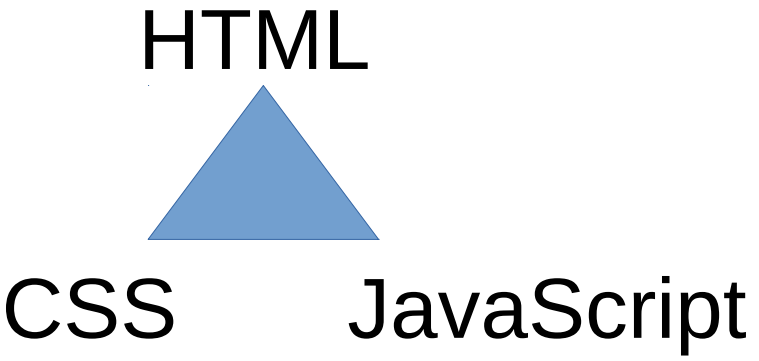
HyperText Markup Language (HTML)
is the standard markup language
for creating web pages and web applications.

Example

```
<!DOCTYPE html>
<html>
<head>
<title>Page Title</title>
</head>
<body>

<h1>My First Heading</h1>
<p>My first paragraph.</p>

</body>
</html>
```

With Cascading Style Sheets (**CSS**), and **JavaScript**, it forms a triad of cornerstone technologies for the World Wide Web

WWW
W3C Technologies =



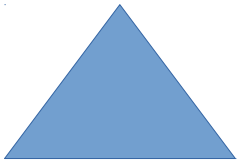
Client(Html+CSS+JS) – Protocol(Http) - Server(*)

Client-Side

Server-Side
expose IP<TCP<HTTP port

HTTP Request (GET,POST,..) URL
headers
body

HTML



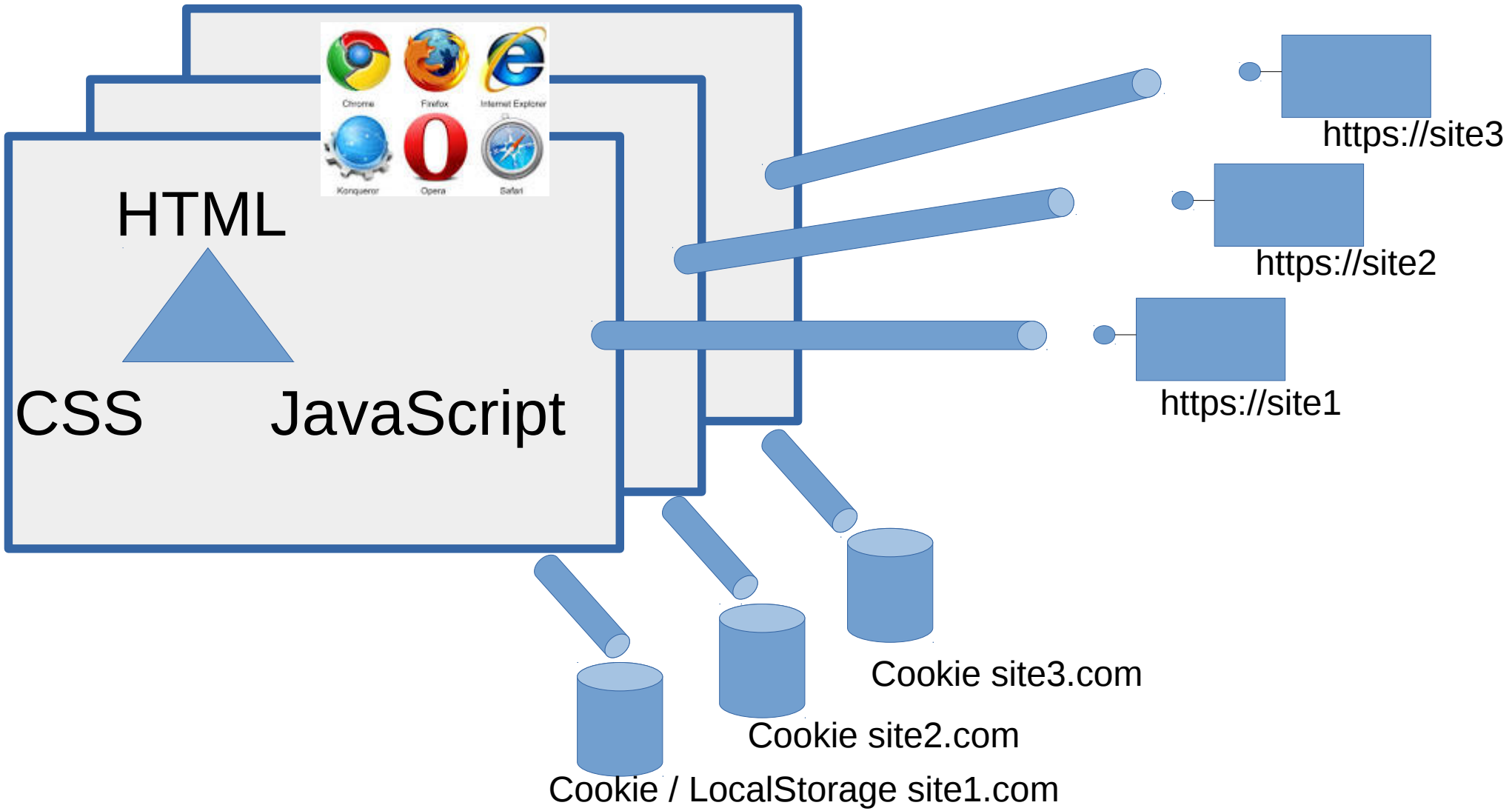
8080
8443

CSS JavaScript

HTTP Response (20x,30x,40x,50x)
headers
body

WebSocket events

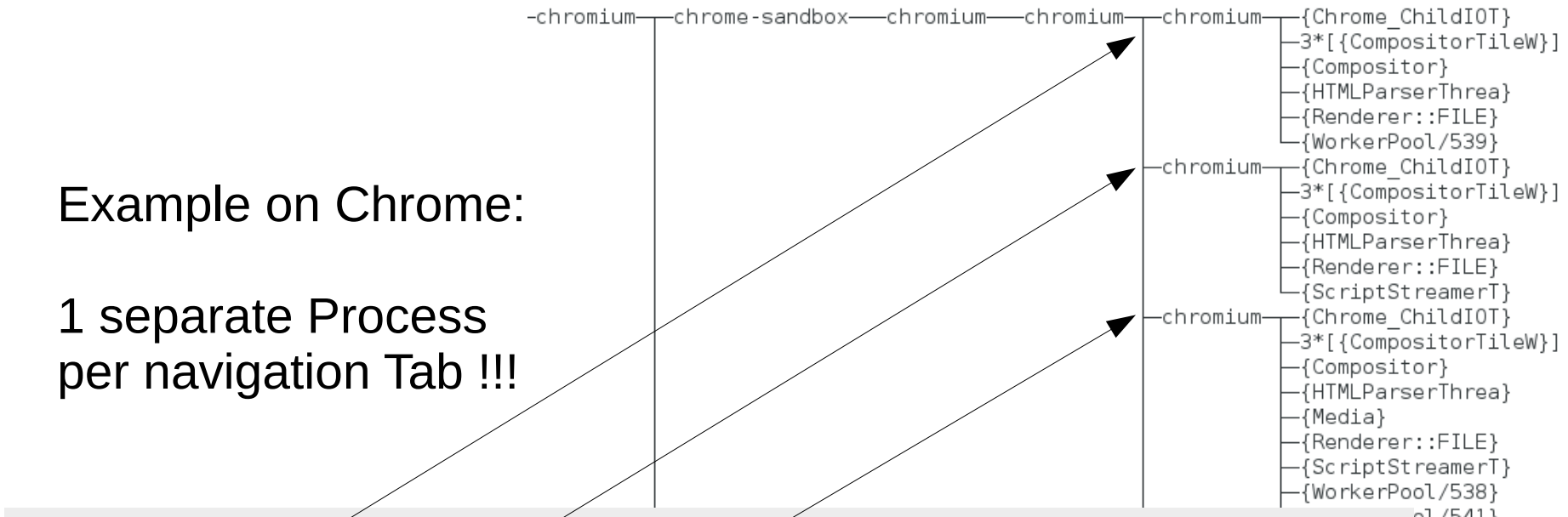
Client-Side = Browser = Security Sandbox



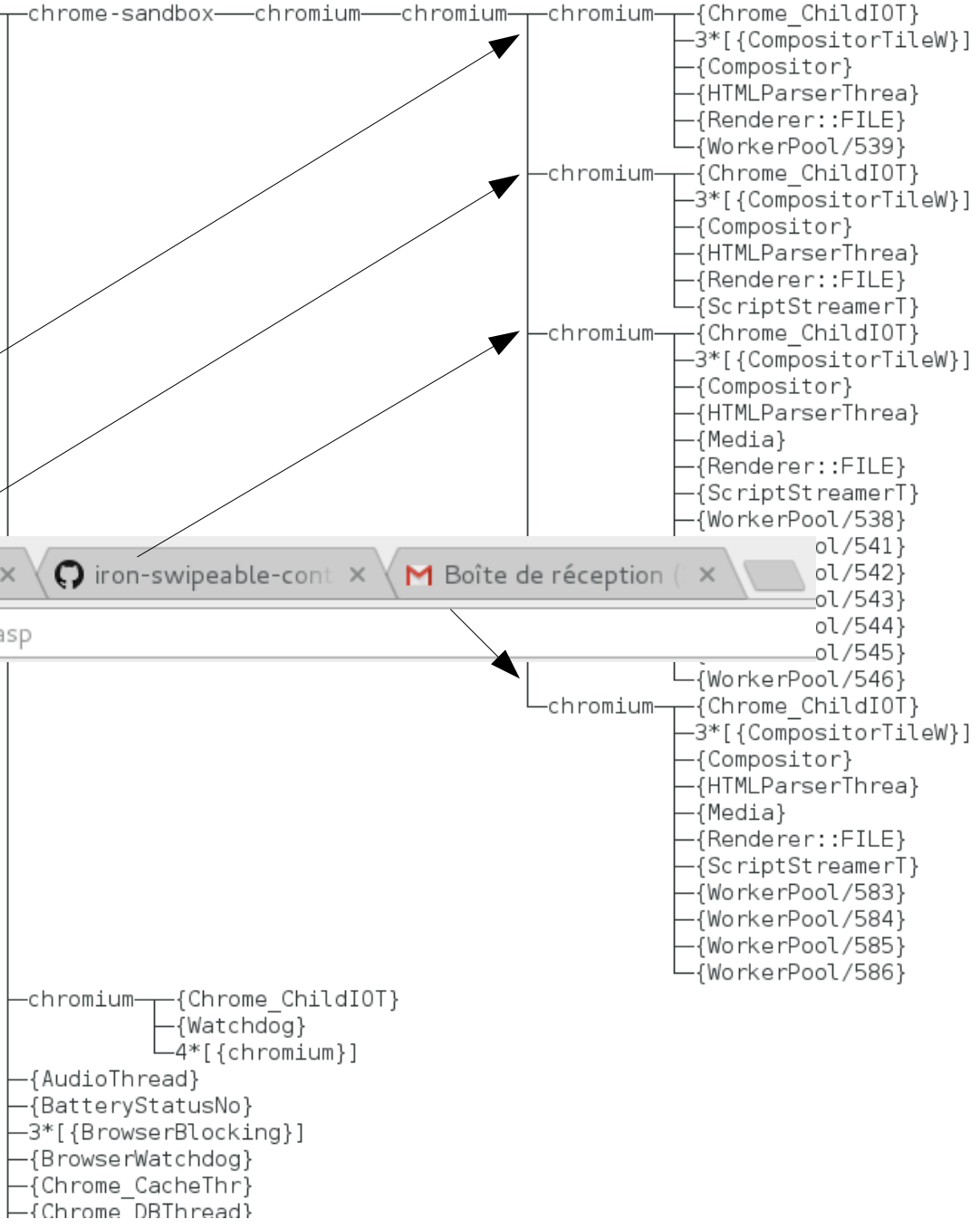
Site Navigation Isolation …

Example on Chrome:

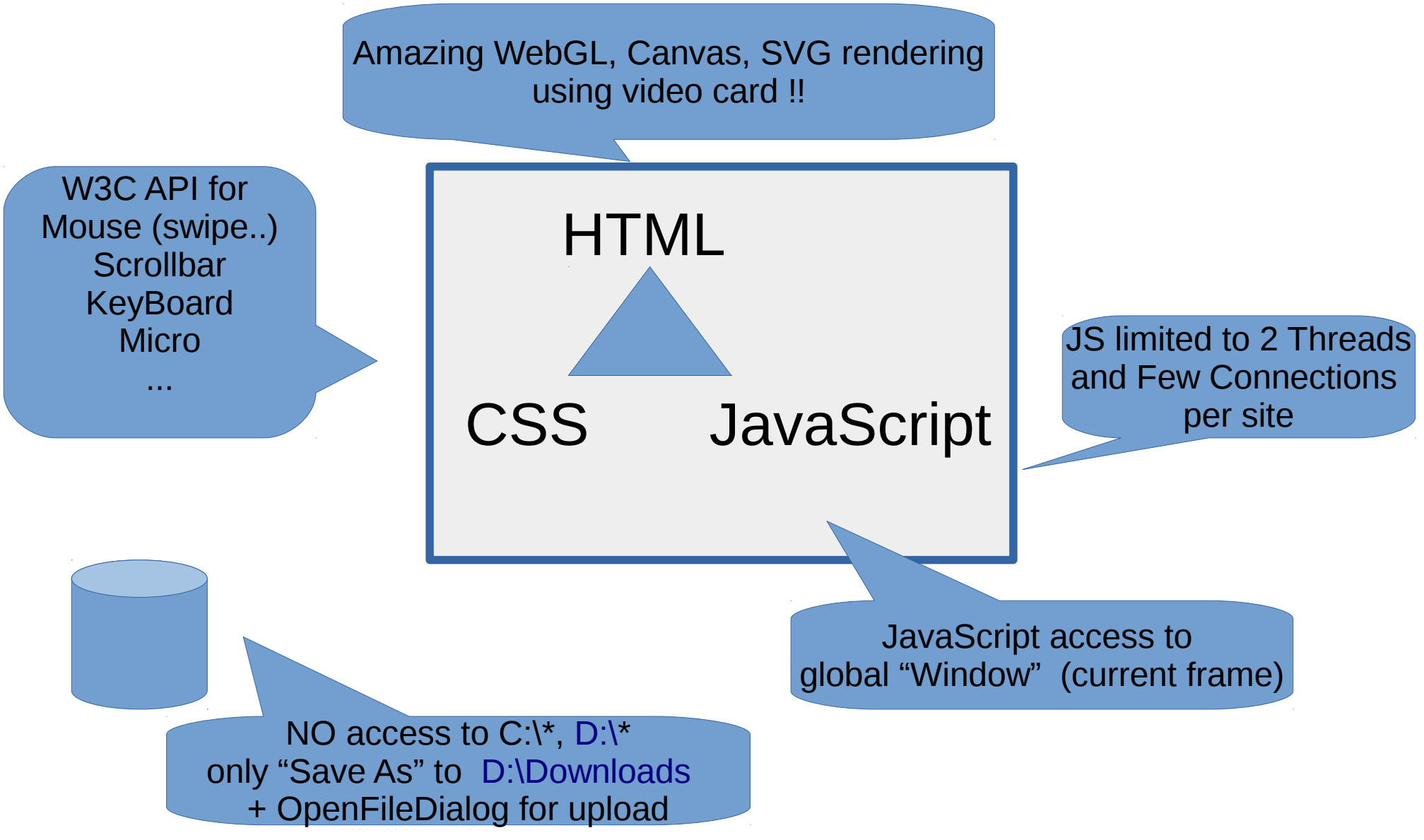
1 separate Process
per navigation Tab !!!

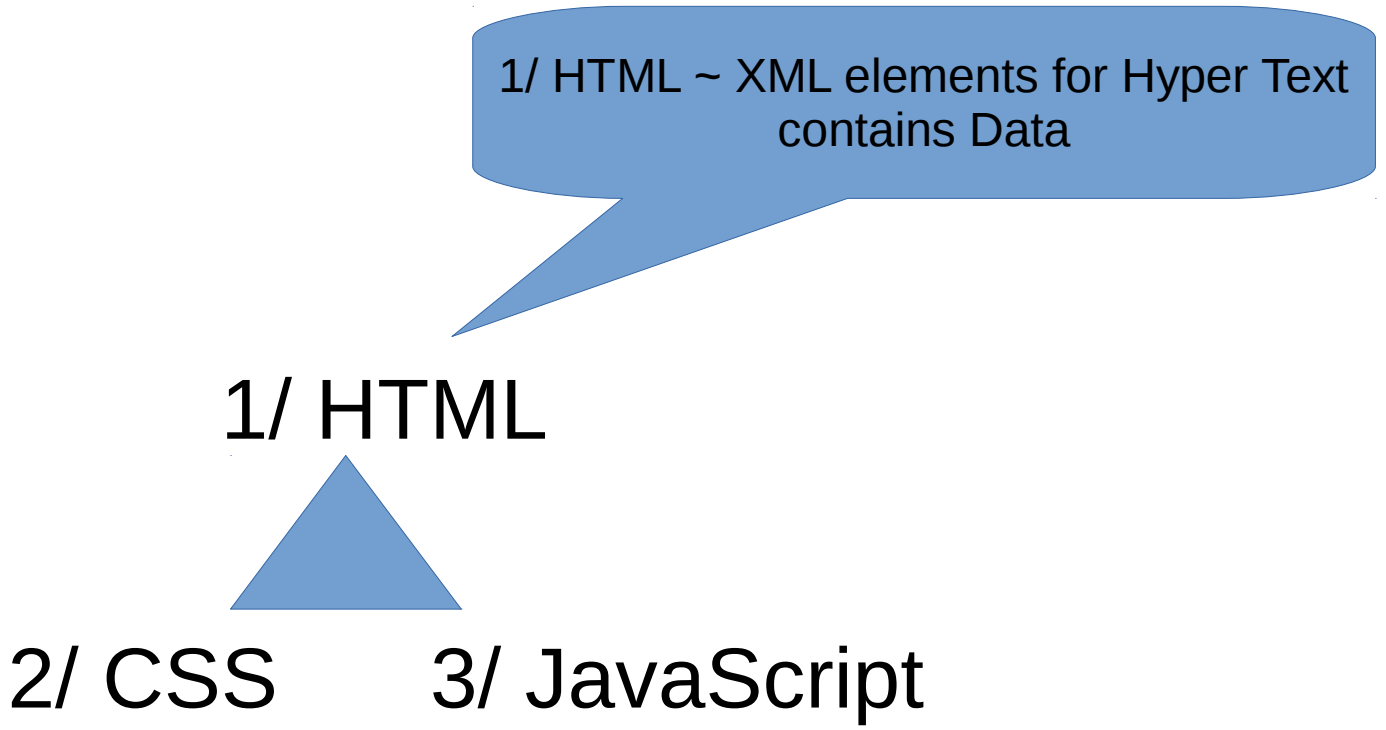


Shared processes
(chrome launcher)

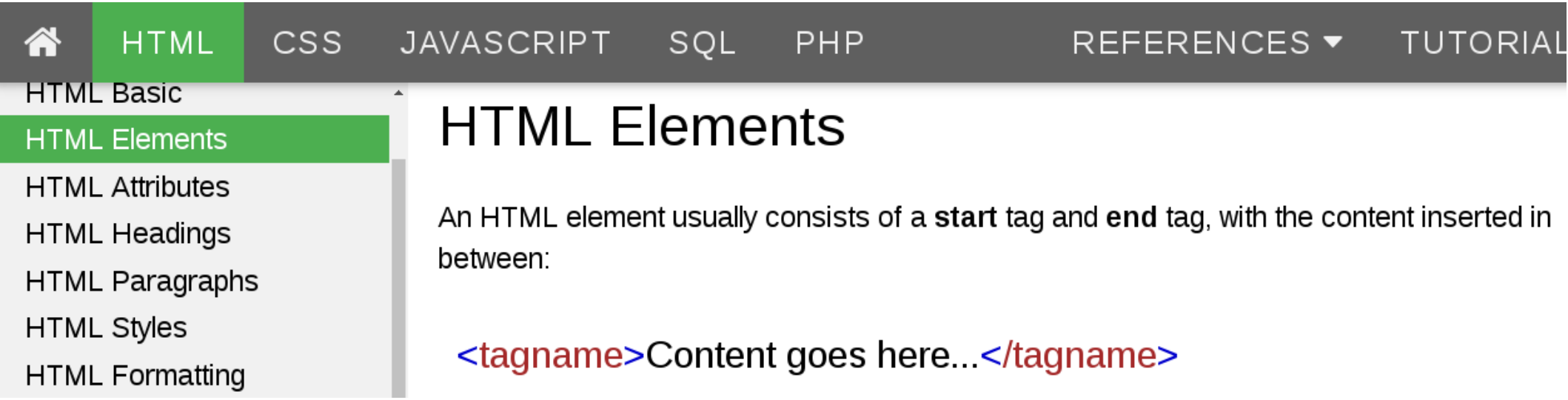


On Client-Side = ONLY
Html / CSS / (Limited) JavaScript




1/ HTML ~ XML elements for Hyper Text
contains Data
1/ HTML
2/ CSS
3/ JavaScript

Element = <tagName> ... </tagName>



<elt attribute1=”value1” attribute2 ../>

Example

```
<a href="http://www.w3schools.com">This is a link</a>
```

Standard W3c Tags

Basic Structure

<h1>Header 1</h1>
<h2>..</h2>
<h3>..<h3>
<h4>..</h4>
paragraph

Text Formatting

<pre> - pre-formatted
<b> - Bold text
 - Important text
<i> - Italic text
 - Emphasized text
<mark> - Marked text
<small> - Small text
<del> - Deleted text
<ins> - Inserted text
<sub> - Subscript text
<sup> - Superscript text

Hyper Link..

<A href=”..”>
<img src=”..”>
<video >
<script >
<iframe>

Other..

<table> <tr><td>

<>
<>...
…
...

Html 1.0 … a simple MarkupLangage for Text

=> many specific tags for text formatting

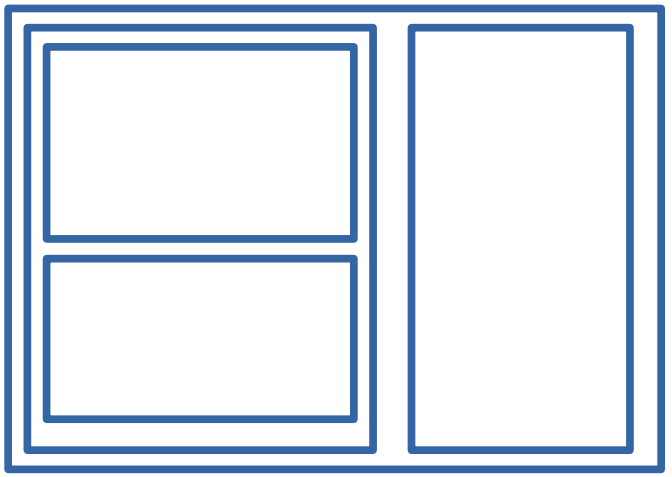
Only 2 families of tag layout: **Blocks** & **Inline**

=> technically no “needs” to add more
than &

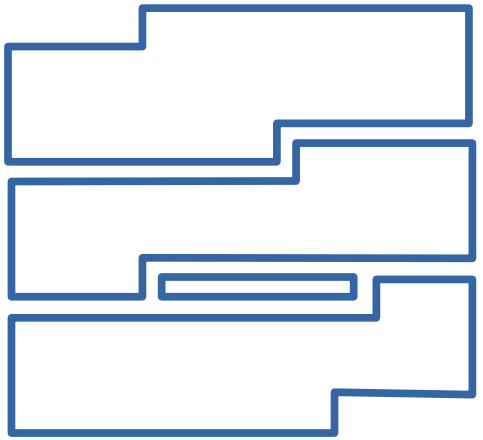
with CSS & JS everywhere

and ..

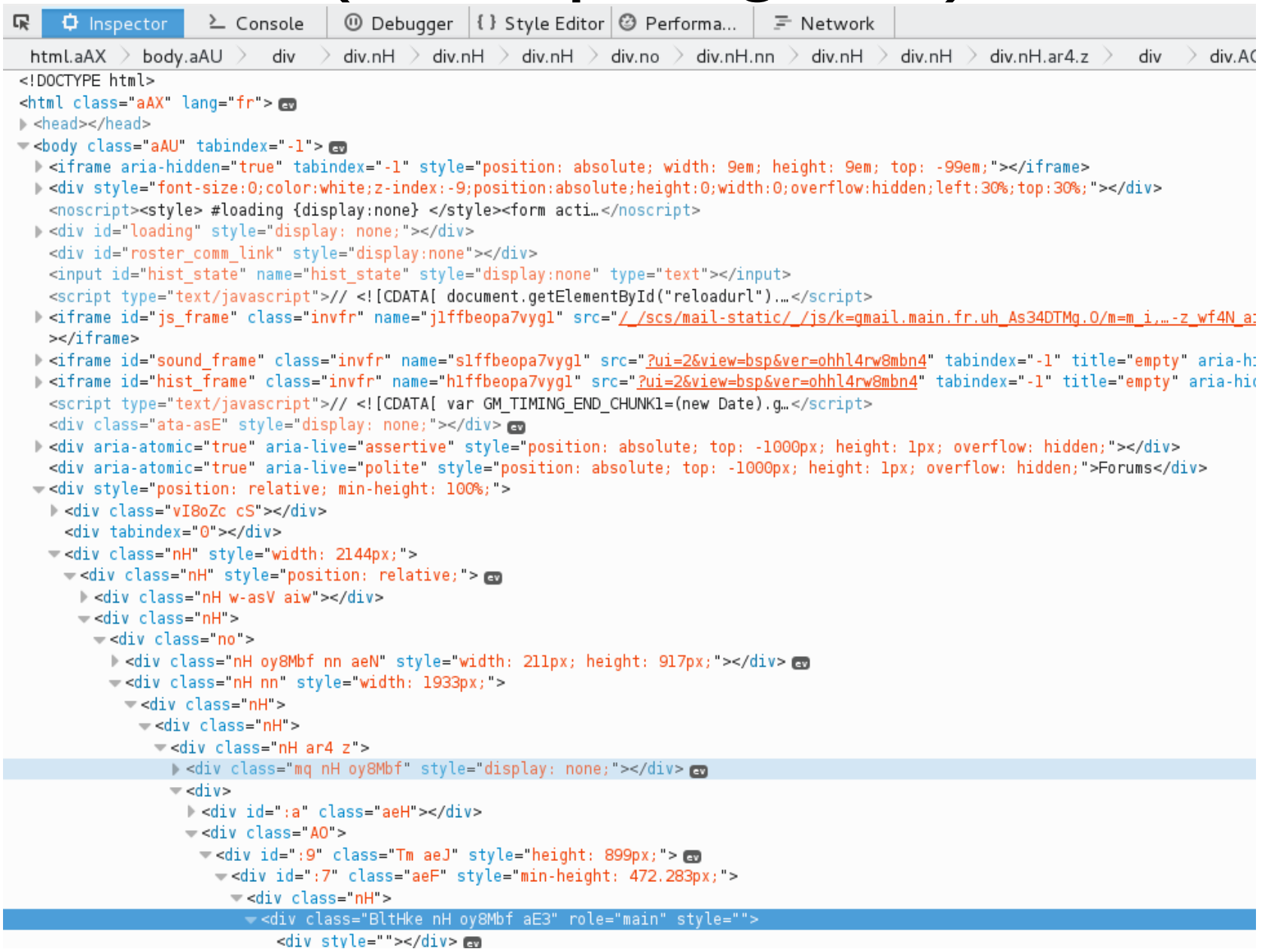
 = blocks



 = inline



Html = soup of (example: gmail)



Code quality with Only technical & ?

Need more semantic tags for web applications

CustomElement

Define your own application specific tags:
<your-custom-tag>

<your-menu> <your-view> <your-object>

Eric Bidelman's article

https://www.html5rocks.com/en/tutorials/webcomponents/customelements/

HOME TUTORIALS UPDATES

Custom Elements

defining new elements in HTML

Table of Contents

- Introduction
- Getting started
- Instantiating elements
- Adding JS properties and methods
- Adding markup
- Styling custom elements
- History and browser support
- Conclusion

Localizations

- □□□
- 中文 (简体)
- 日本語
- Contribute another

By Eric Bidelman

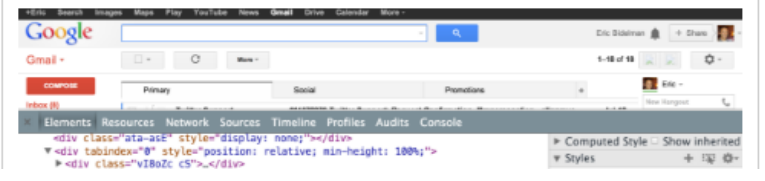
Published: August 28th, 2013
Updated: October 3rd, 2016
Comments: 0

Heads up!

This article describes an old version of the Custom elements API (v0). If you're interested creating custom elements, check out our new article at developers.google.com, "Custom elements v1: reusable web components". It covers everything in the newer Custom elements v1 spec.

Introduction

The web severely lacks expression. To see what I mean, take a peek at a "modern" web app like GMail:



Modern web apps: built with soup.

There's nothing modern about `<div>` soup. And yet,…this is how we build web apps. It's sad. Shouldn't we demand more from our platform?

Sexy markup. Let's make it a thing.

HTML gives us an excellent tool for structuring a document but its vocabulary is limited to elements the HTML standard defines.

What if the markup for GMail wasn't atrocious? What if it was beautiful:

```
<hangout-module>
  <hangout-chat from="Paul, Addy">
    <hangout-discussion>
      <hangout-message from="Paul" profile="profile.png"
          profile="118075919496626375791" datetime="2013-07-17T12:02">
        <p>Feelin' this Web Components thing.</p>
        <p>Heard of it?</p>
      </hangout-message>
    </hangout-discussion>
  </hangout-chat>
  <hangout-chat>...</hangout-chat>
</hangout-module>
```

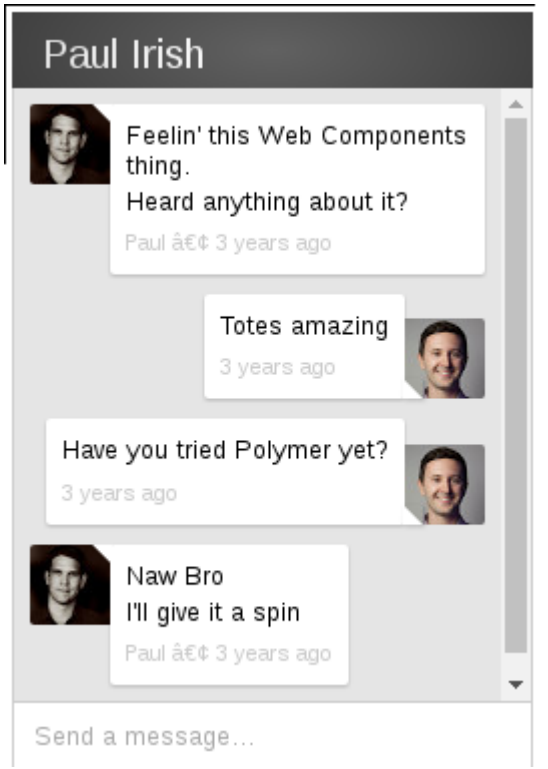
Try the demo!

How refreshing! This app totally makes sense too. It's **meaningful**, **easy to**

Example Demo :

http://html5-demos.appspot.com/hangouts

```
<!DOCTYPE html>
<html>
▶<head>…</head>
▼<body>
  ▶<div class="newchat">…</div>
  ▼<div id="hangouts">
      <!-- <hangout-module from="Paul Irish"
      profile="118075919496626375791"></hangout-module> -->
    ▼<hangout-module>
      ▼#shadow-root (open)
        ▶<style>…</style>
        ▶<hangout-header>…</hangout-header>
        ▼<hangout-discussion id="discussion">
          ▶#shadow-root (open)
          ▶<template repeat="{{message in messages}}">…</template>
          ▼<hangout-message isother="{{message.isother}}" from="{{from}}"
          profile="{{message.profile}}" datetime="{{message.datetime}}"
          now="{{now}}" class="isother">
            ▶#shadow-root (open)
            ▶<template repeat="{{msg in message.msg}}">…</template> == $0
              <p>Feelin' this Web Components thing.</p>
              <p>Heard anything about it?</p>
            </hangout-message>
          ▶<hangout-message isother="{{message.isother}}" from="{{from}}"
          profile="{{message.profile}}" datetime="{{message.datetime}}"
          now="{{now}}" class="self">…</hangout-message>
          ▶<hangout-message isother="{{message.isother}}" from="{{from}}"
          profile="{{message.profile}}" datetime="{{message.datetime}}"
          now="{{now}}" class="self">…</hangout-message>
          ▶<hangout-message isother="{{message.isother}}" from="{{from}}"
          profile="{{message.profile}}" datetime="{{message.datetime}}"
          now="{{now}}" class="isother">…</hangout-message>
```



W3C Working Draft

TABLE OF CONTENTS

1. **Conformance**

2. **HTML: Custom elements**
2.1 Introduction
2.1.1 Creating an autonomous custom element
2.1.2 Creating a customized built-in element
2.1.3 Drawbacks of autonomous custom elements
2.1.4 Upgrading elements after their creation
2.2 Requirements for custom element constructors
2.3 Core concepts
2.4 The CustomElementRegistry interface
2.5 Upgrades
2.6 Custom element reactions

3. **HTML: HTML element constructors**

4. **Miscellaneous patches**
4.1 HTML: The `Window` object
4.2 HTML: Pseudo-classes
4.3 HTML: Creating and inserting nodes
4.4 HTML: Parsing XHTML documents
4.5 HTML: Content model
4.6 HTML: Wide-ranging patches
4.7 DOM: Elements
4.8 DOM: Cloning
4.9 DOM: Mutation algorithms
4.10 DOM: Wide-ranging patches

A. **Acknowledgments**

B. **References**
B.1 Normative references

Custom Elements

W3C Working Draft 13 October 2016

This version:
https://www.w3.org/TR/2016/WD-custom-elements-20161013/

Latest published version:
https://www.w3.org/TR/custom-elements/

Latest editor's draft:
https://w3c.github.io/webcomponents/spec/custom/

Previous version:
https://www.w3.org/TR/2016/WD-custom-elements-20161002/

Editor:
Domenic Denicola, Google, Inc.

Repository:
We are on Github.
File a bug.
Commit history.

Mailing list:
public-webapps@w3.org

Implementation:
Can I use Custom Elements?
Test Suite
Test Suite repository

Copyright © 2016 W3C® (MIT, ERCIM, Keio, Beihang). W3C liability, trademark and permissive document license rules apply.

Abstract

This specification describes the method for enabling the author to define and use new types of DOM elements in a document. It is a copy of the relevant parts of [HTML] up through commit

Custom Elt

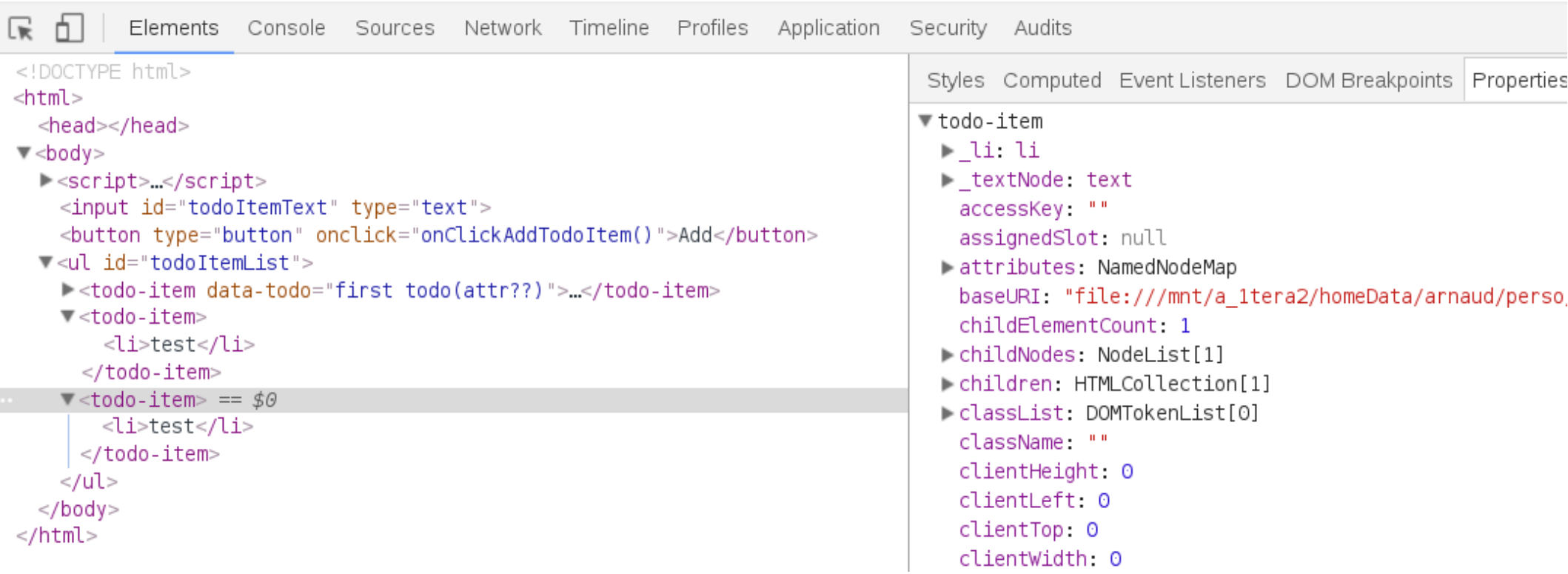
```
<input id="todoItemText" type="text"/>
<button type="button"
    onclick="onClickAddTodoItem()"
    >Add</button>
<ul id="todoItemList">
    <todo-item
        todo="first todo(attr??)" />
</ul>
```

```
"use strict";
class TodoItem extends HTMLElement {
    constructor(self) {
        super(self);
        console.info("ctor");
    }
    createdCallback() {
        console.info("created");
        this._li = document.createElement("li");
        this._textNode = document.createTextNode("");
        this._li.appendChild(this._textNode);
        this.appendChild(this._li);
    }
    attachedCallback() { console.info("attached"); }
    detachedCallback() {
        console.info("detached");
    }
    set todo(text) {
      console.info("set attribute");
      this._textNode.data = text;
    }
    get todo() {
      return this._textNode.data;
    }
    attributeChangedCallback(name, oldValue, newValue) {
      if (name === 'todo') {
        console.info("attributeChange");
        this._textNode.data = newValue;
      }
    }
}
document.registerElement("todo-item", TodoItem);

function findById(id) { return document.getElementById(id); }
function onClickAddTodoItem() {
  let text = findById('todoItemText').value;
  let newTodoItemElt = document.createElement("todo-item");
  newTodoItemElt.todo = text;
  findById('todoItemList').appendChild(newTodoItemElt);
}
```

Custom Elements ...

attribute not working in this s: Add

-
- test
- attribute not working in this sample??



WebComponent
… Simplify CustomElement

Goals :

1) simplify Javascript ...replace by Html
   <dom-module><template>

2) supports old browser via PolyFill

3) use Template + ShadowDom

Please help us make Web Components & PWA's better by completing this survey

WebComponents.org

a place to discuss and evolve web component best-practices

POLYFILLS

The webcomponent.js polyfills enable Web Components in (evergreen) browsers that lack native support.

Install with Bower

```
bower install webcomponentsjs
```

Install with npm

```
npm install webcomponents.js
```

Download webcomponents.js

0.7.12 (117KB minified, 34KB gzipped)

learn more about the polyfills

BROWSER SUPPORT

CHROME OPERA FIREFOX SAFARI IE/EDGE

DISCOVER

CUSTOMELEMENTS.IO
A gallery to display Web Components created by the community.

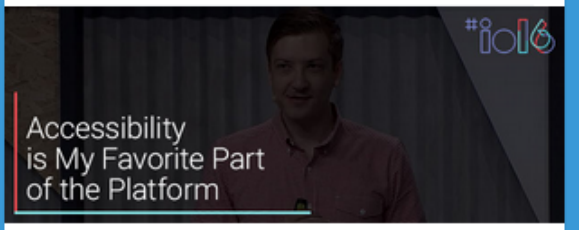
POLYMER CATALOG
The official product line catalog for Polymer Web Components.

BUILT WITH POLYMER
A curated collection of web apps and websites using Polymer.

SPECS

CUSTOM ELEMENTS
This specification describes the method for enabling the author

PRESENTATIONS



ACCESSIBILITY IS MY FAVORITE PART OF THE PLATFORM

My favorite part of my job is when I get to work on accessibility. I see this as removing uncertainty and anxiety from people's day, and helping them access the information they need. But working with accessibility can be frustrating, especially when you feel like you're fighting the platform instead of working with it.

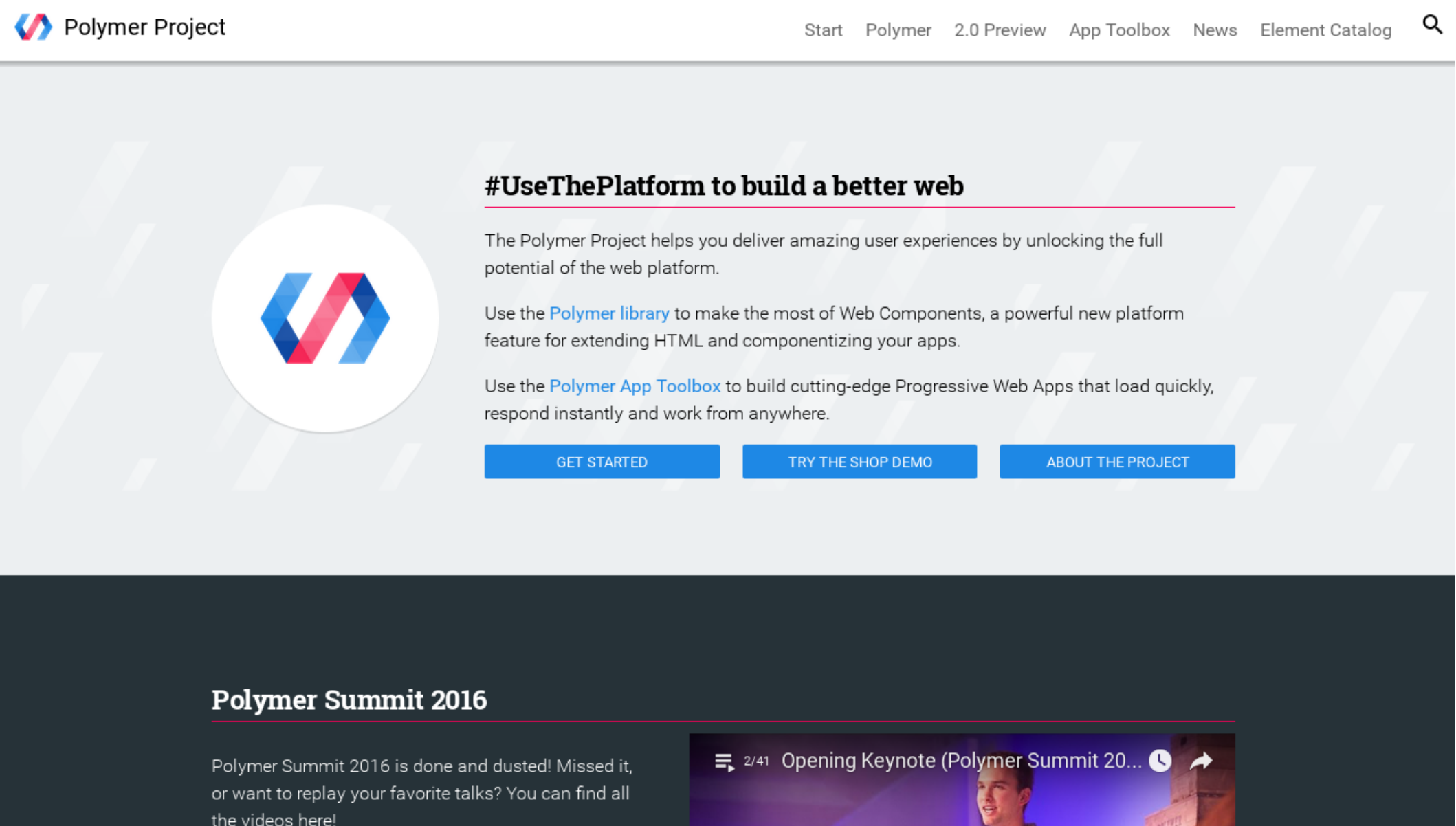
Read More >

see all presentations

WebComponents = 4 w3c specs

= CustomElement + Import + Template + ShadowDom

www.polymer-project.org
= Google Library of WebComponents

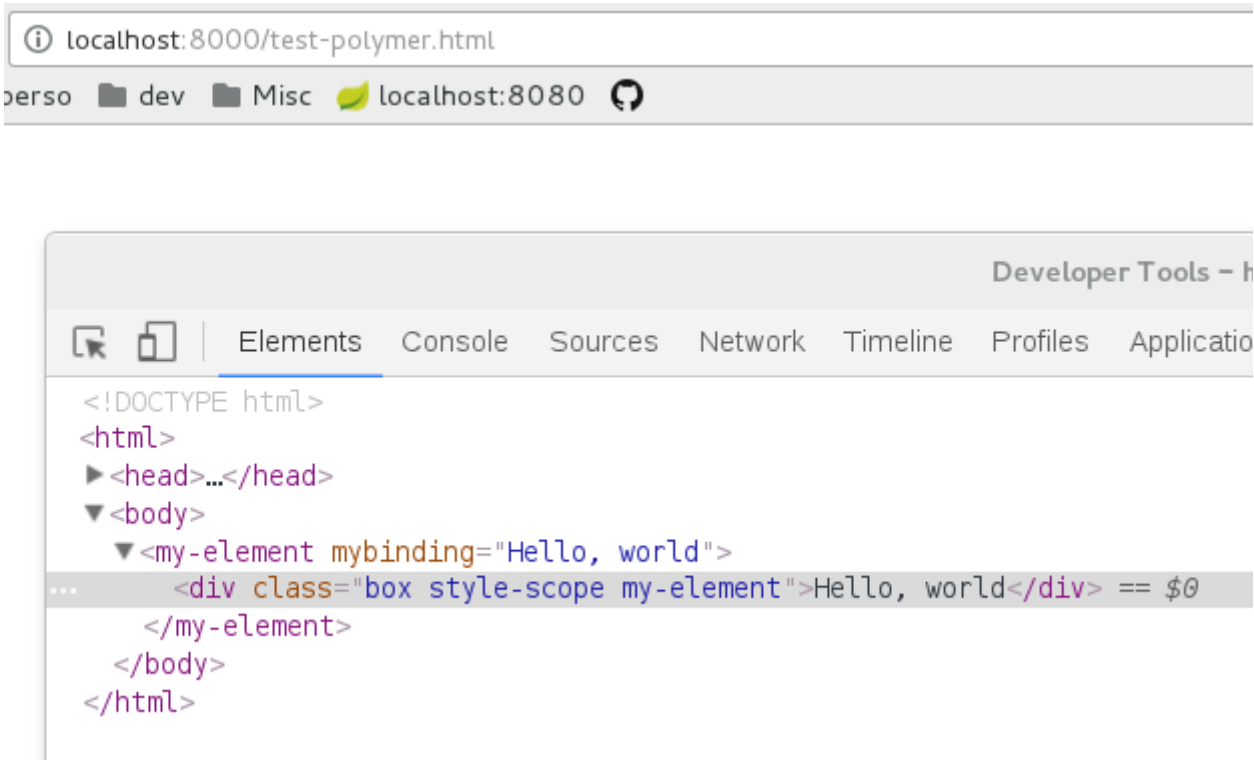


Polymer example

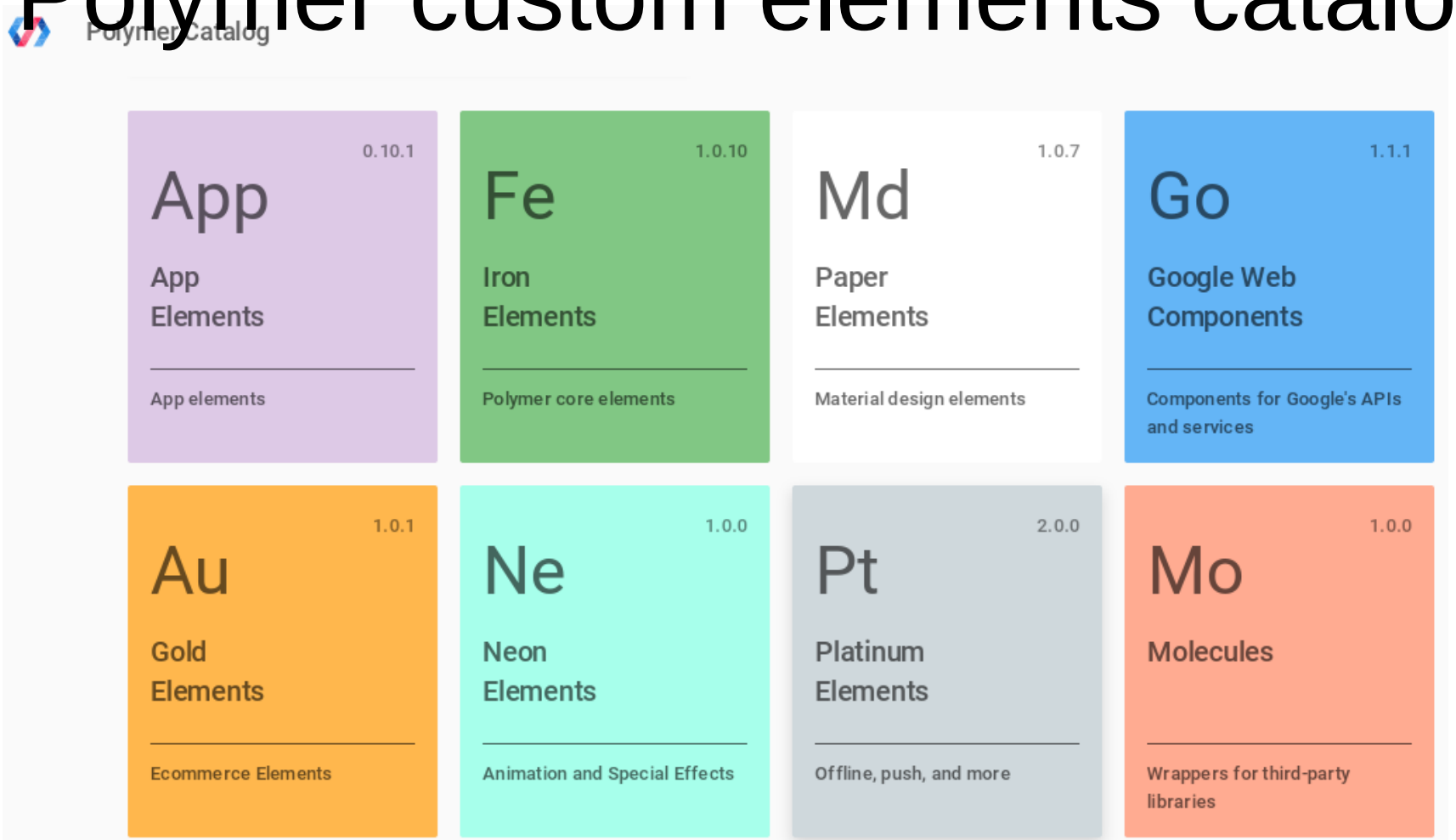
```
<!doctype html>
<html>
<head>
    <meta charset="utf-8">
    <script src="bower_components/webcomponentsjs/webcomponents-lite.js"></script>
    <link rel="import" href="bower_components/polymer/polymer.html">
    <link rel="import" href="my-element.html" />
    <!-- notice... does not work on file:// ... need use server, example: "python -m SimpleHTTPServer
</head>
<body>
    <my-element mybinding="Hello, world"></my-element>
</body>
</html>
```

File “my-element.html”

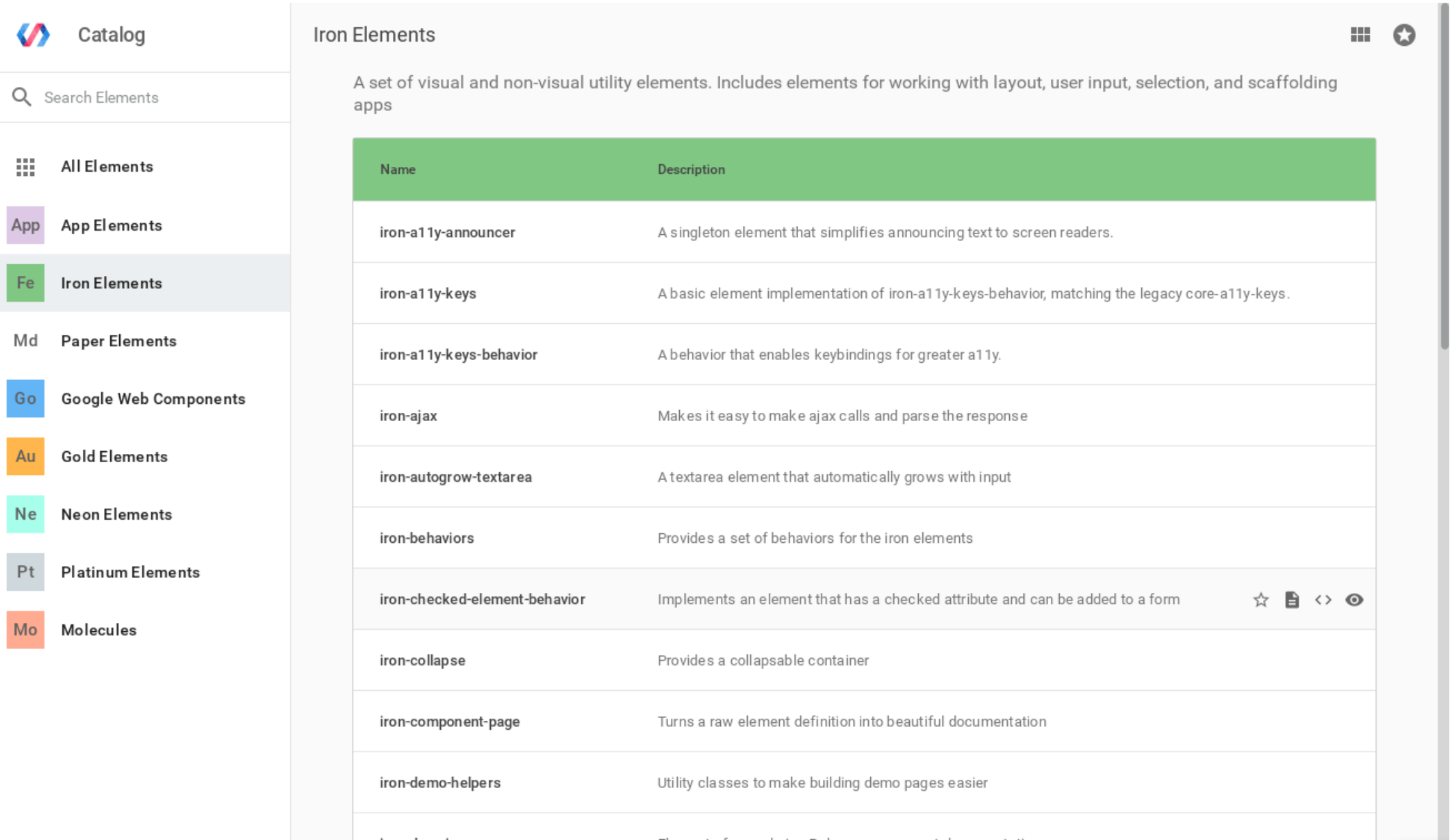
```
<dom-module id="my-element">
  <template>
    <style>
    .box{
      color: grey;
    }
    </style>
    <div class="box">{{mybinding}}</div>
    </template>
    <script>
      Polymer({
        is: "my-element",
        properties: {
          mybinding: {
            type: String,
            value: "My text here"
          }
        }
      });
    </script>
</dom-module>
```



Polymer custom elements catalog

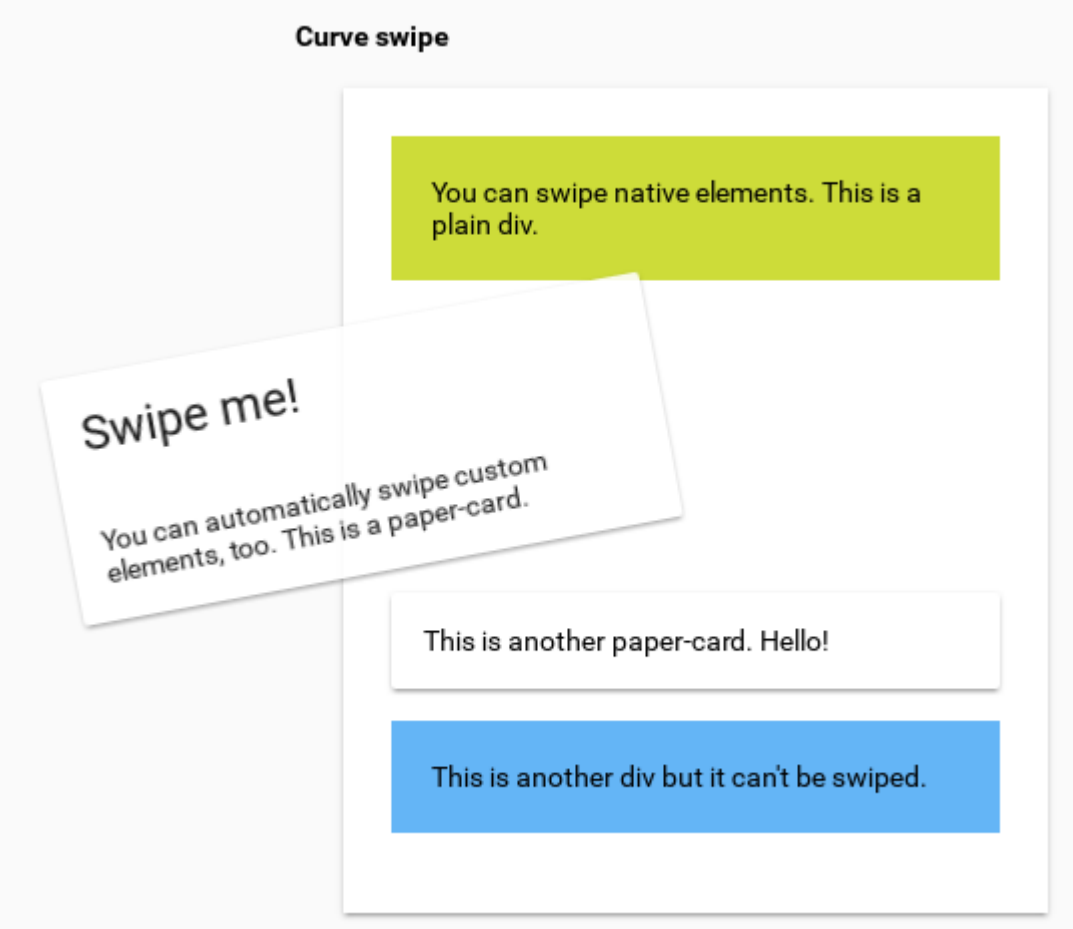


Polymer Core Elements : Fe (Iron)



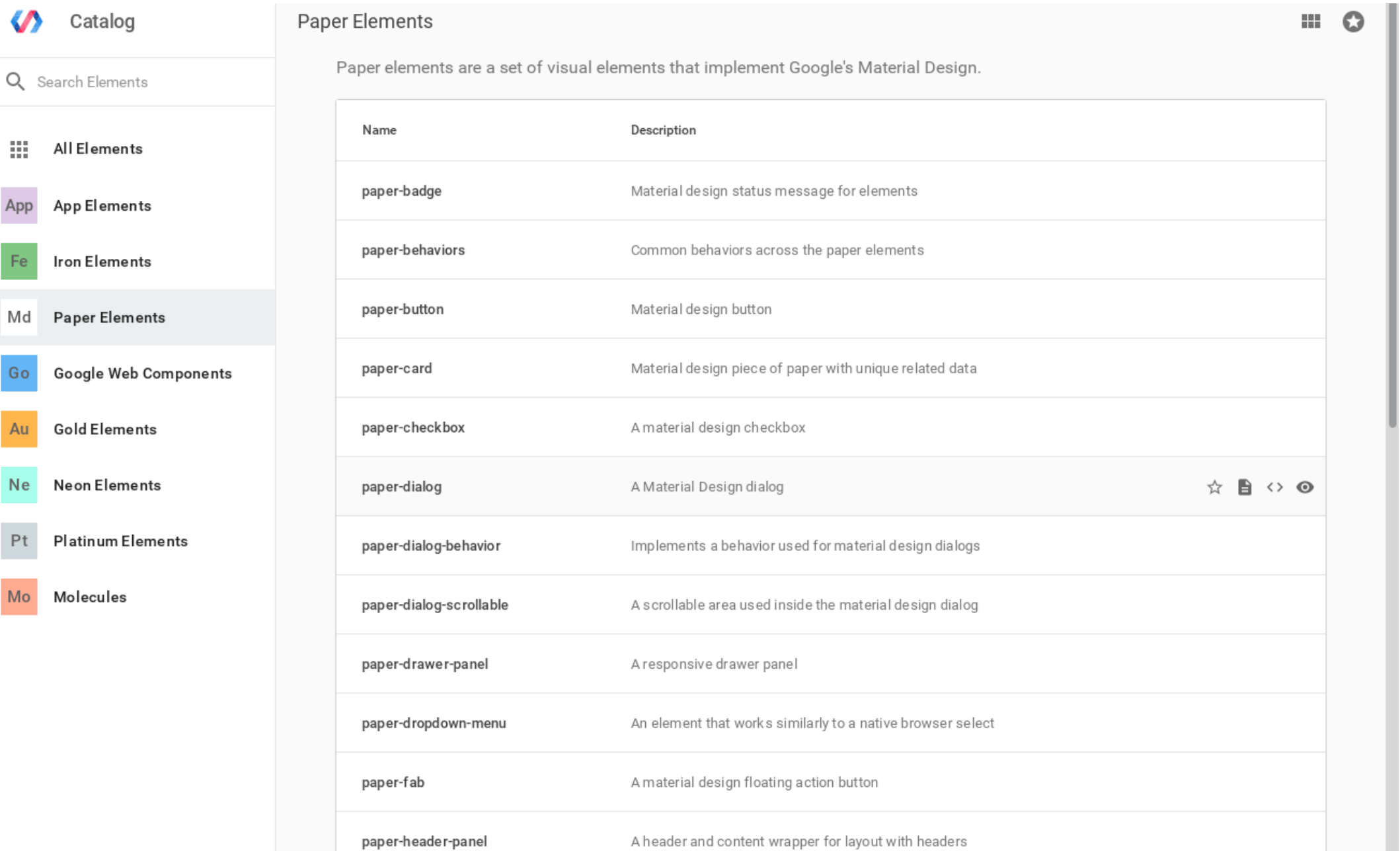
Iron example <iron-swipeable-container>

https://github.com/PolymerElements/Iron-swipeable-container/blob/master/demo/index.html

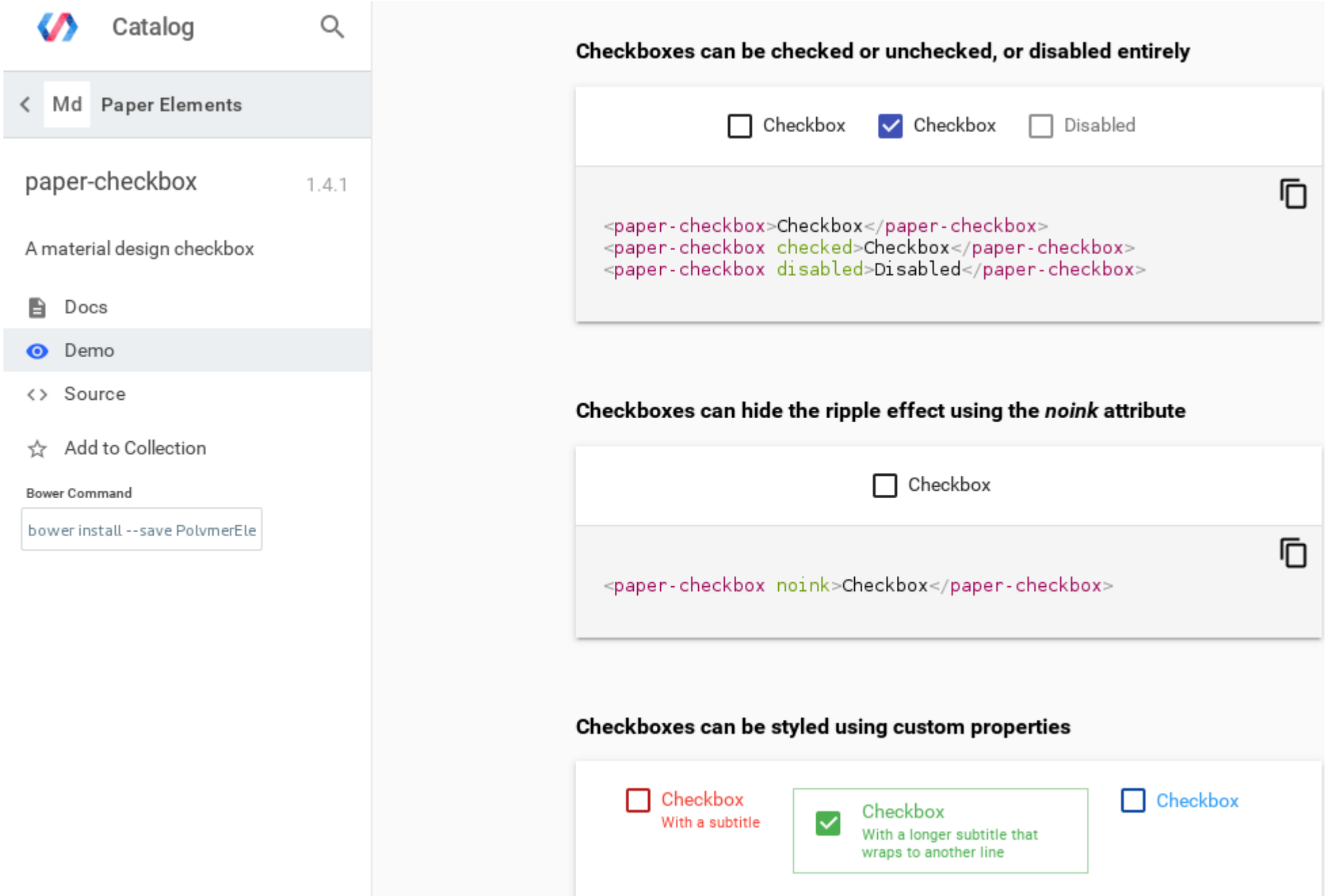


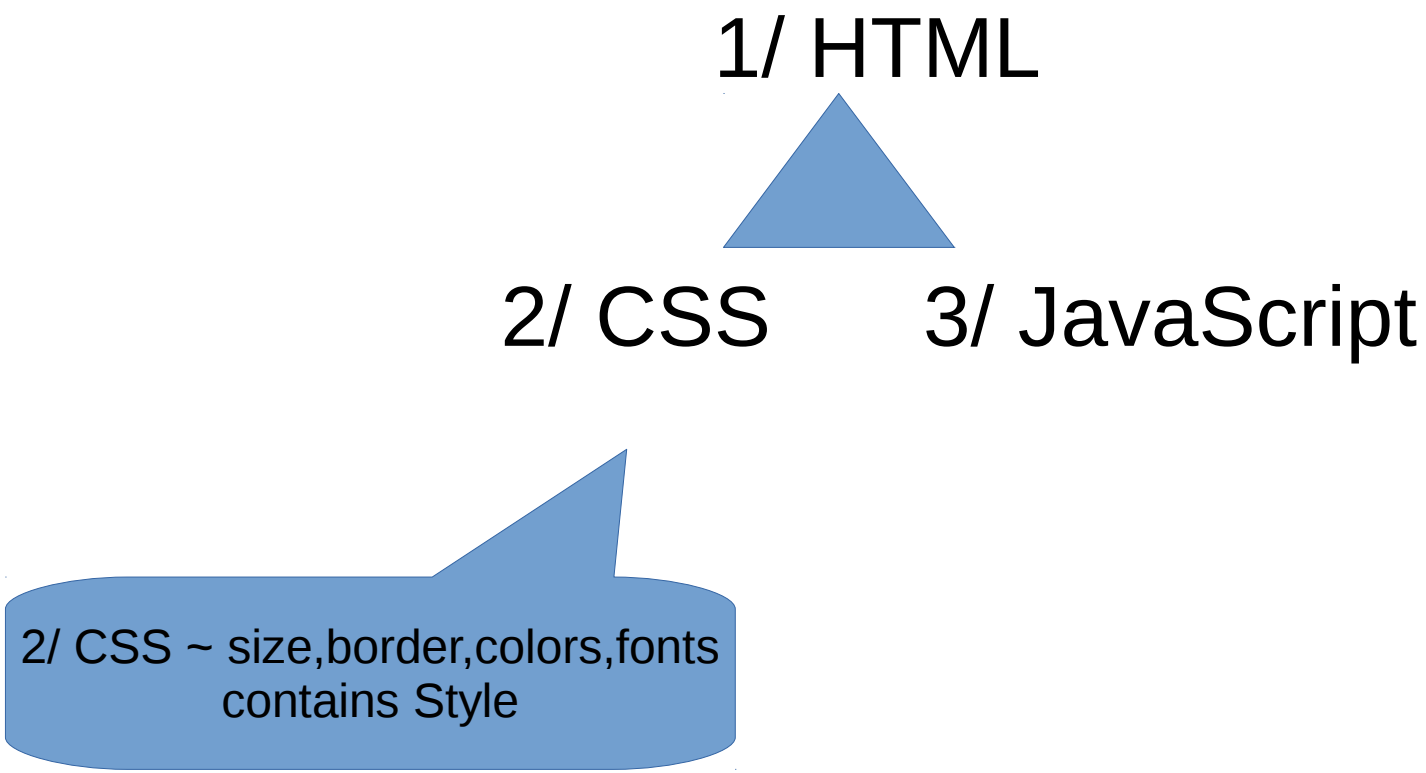

```
<h4>Curve swipe</h4>
<div class="vertical-section">
  <iron-swipeable-container swipe-style="curve">
    <div class="item swipe">You can swipe native

    <paper-card class="swipe" heading="Swipe me!">
      <div class="card-content">
        You can automatically swipe custom elements, too. This is a paper-card.
      </div>
    </paper-card>
```

Material Design : Md (Paper)



Md Paper Example




1/ HTML
2/ CSS
3/ JavaScript
2/ CSS ~ size,border,colors,fonts
contains Style

style=” …inlined CSS property:value;”

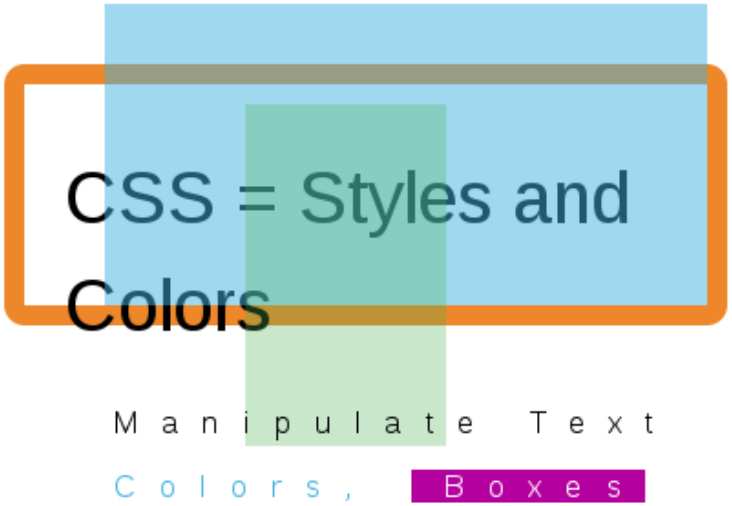
Styles … using CSS files

HTML CSS
HTML Links
HTML Images
HTML Tables
HTML Lists
HTML Blocks
HTML Classes
HTML Iframes
HTML JavaScript
HTML Head
HTML Layout
HTML Responsive
HTML Entities
HTML Symbols
HTML Charset
HTML URL Encode
HTML XHTML

HTML Forms
HTML Forms
HTML Form Elements
HTML Input Types

HTML Styles - CSS

❮ Previous



Styling HTML with CSS

CSS stands for **C**ascading **S**tyle **S**heets.

CSS describes **how HTML elements are to be displayed on screen, paper, or in other media**.

Internal / External CSS

Internal CSS

An internal CSS is used to define a style for a single HTML page.

An internal CSS is defined in the <head> section of an HTML page, within a <style> element:

Example

```
<!DOCTYPE html>
<html>
<head>
<style>
body {background-color: powderblue;}
h1   {color: blue;}
p    {color: red;}
</style>
```

External CSS

An external style sheet is used to define the style for many HTML pages.

With an external style sheet, you can change the look of an entire web site, by changing one file!

To use an external style sheet, add a link to it in the <head> section of the HTML page:

Example

```
<!DOCTYPE html>
<html>
<head>
  <link rel="stylesheet" href="styles.css">
```

Style with < class=”..”>

HTML Classes
HTML Iframes
HTML JavaScript
HTML Head
HTML Layout
HTML Responsive
HTML Entities
HTML Symbols
HTML Charset
HTML URL Encode
HTML XHTML

HTML Forms
HTML Forms
HTML Form Elements
HTML Input Types
HTML Input Attributes

HTML5
HTML5 Intro
HTML5 Support
HTML5 Elements
HTML5 Semantics
HTML5 Migration
HTML5 Style Guide

HTML Graphics
HTML Canvas

HTML The class Attribute

❮ Previous

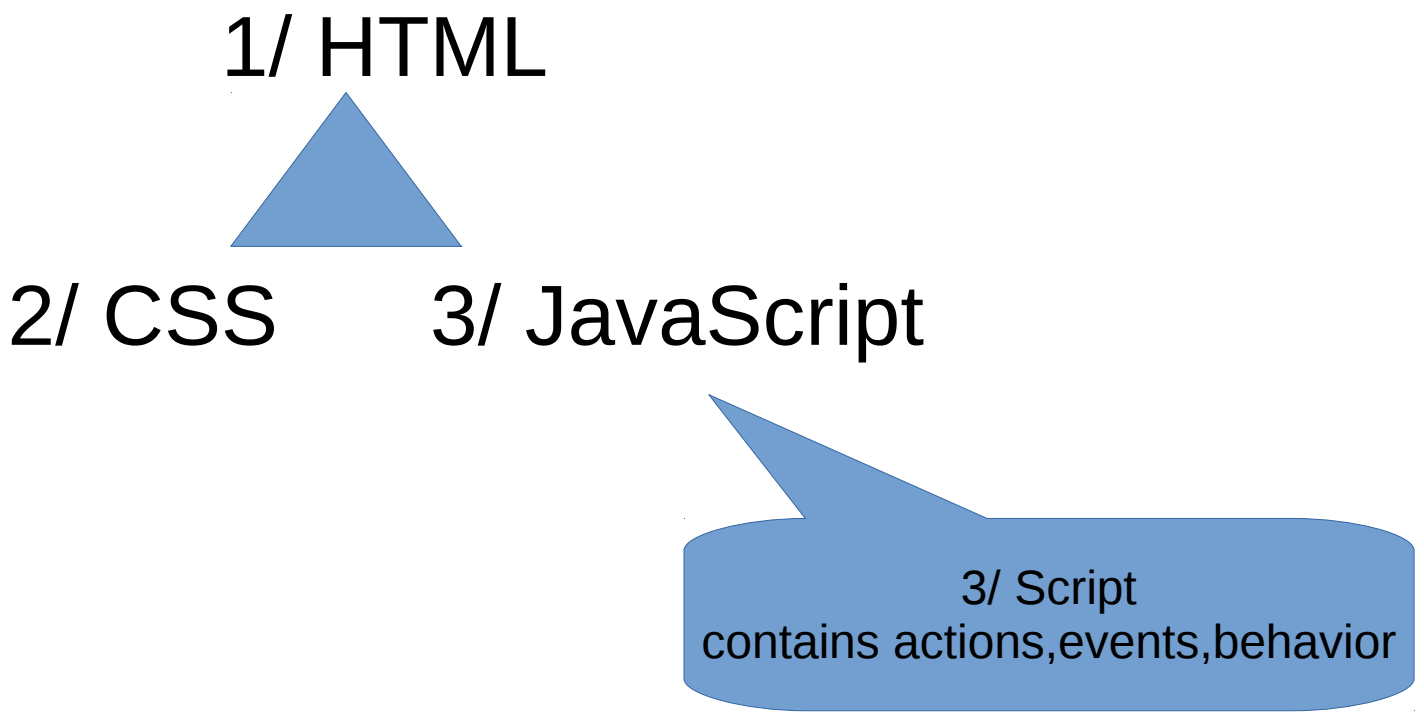
Using The class Attribute

The HTML class attribute makes it possible to define equal styles for elements with the same class name.

Here we have three elements that points to the same class name:

Example

```
<!DOCTYPE html>
<html>
<head>
<style>
div.cities {
    background-color: black;
    color: white;
    margin: 20px 0 20px 0;
    padding: 20px;
}
</style>
</head>
<body>

<div class="cities">
<h2>London</h2>
```


1/ HTML
2/ CSS
3/ JavaScript
3/ Script
contains actions,events,behavior

HTML JavaScript
HTML Head
HTML Layout
HTML Responsive
HTML Entities
HTML Symbols
HTML Charset
HTML URL Encode
HTML XHTML

HTML Forms
HTML Forms
HTML Form Elements
HTML Input Types
HTML Input Attributes

HTML5
HTML5 Intro
HTML5 Support
HTML5 Elements
HTML5 Semantics
HTML5 Migration
HTML5 Style Guide

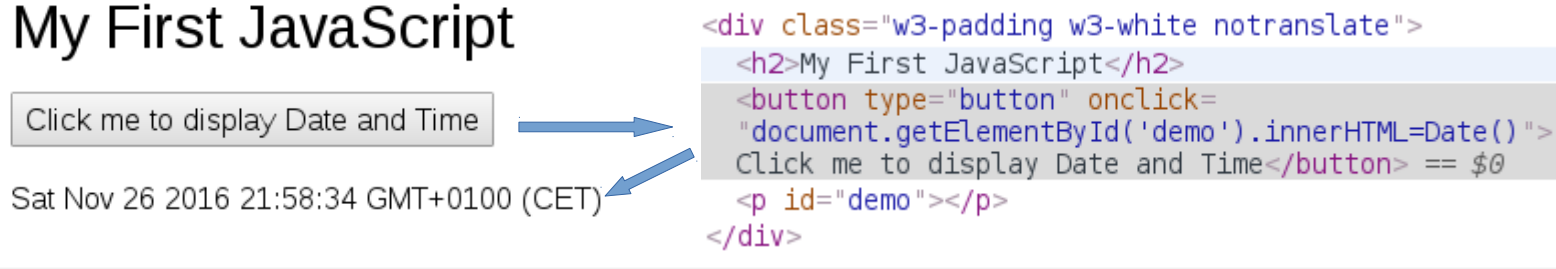
HTML Graphics
HTML Canvas
HTML SVG
HTML Google Maps

HTML JavaScript

❮ Previous

JavaScript makes HTML pages more dynamic and interactive.

Example



Try it Yourself »

The HTML <script> Tag

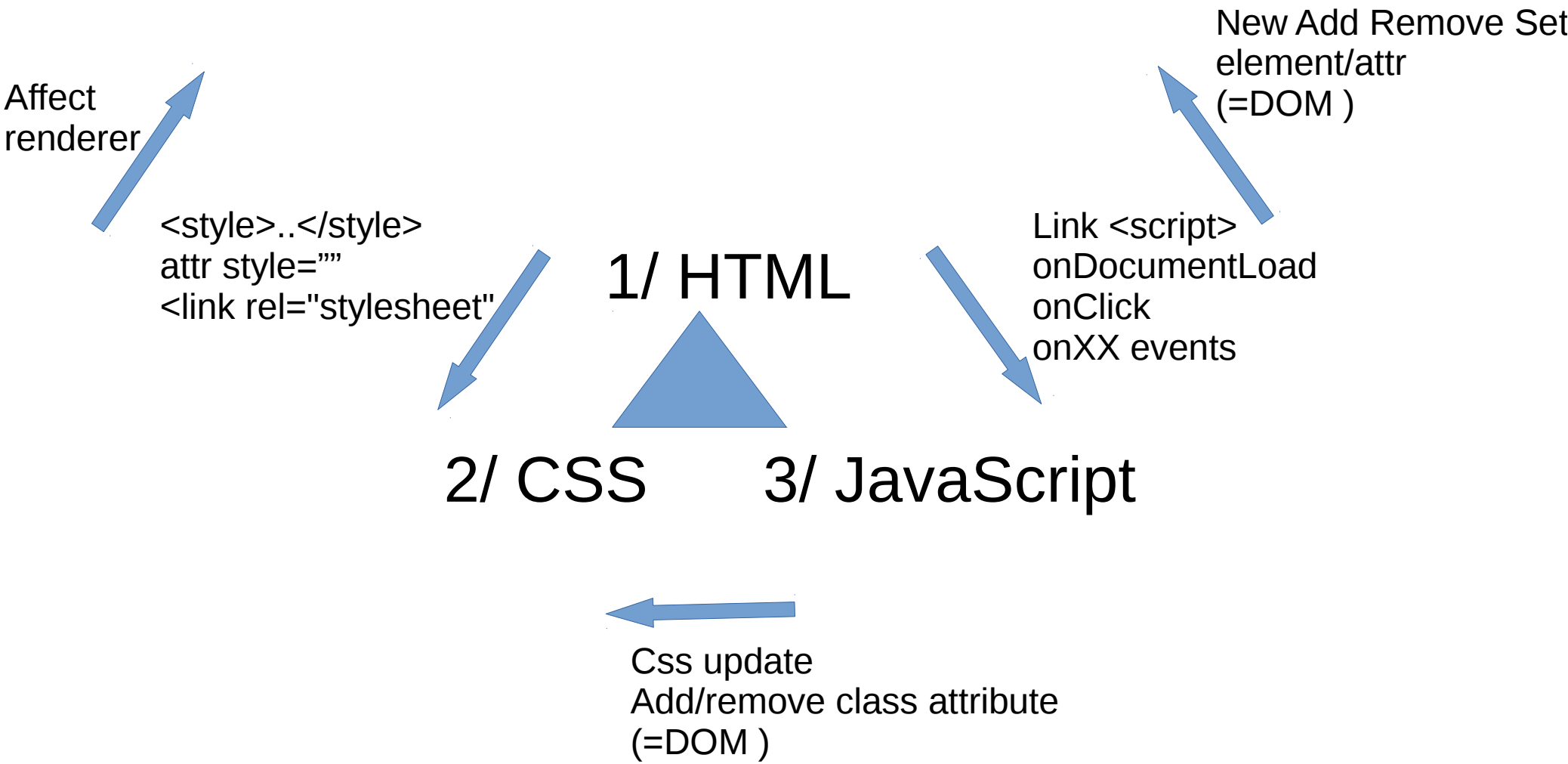
The **<script>** tag is used to define a client-side script (JavaScript).

The <script> element either contains scripting statements, or it points to an external script file through the **src** attribute.

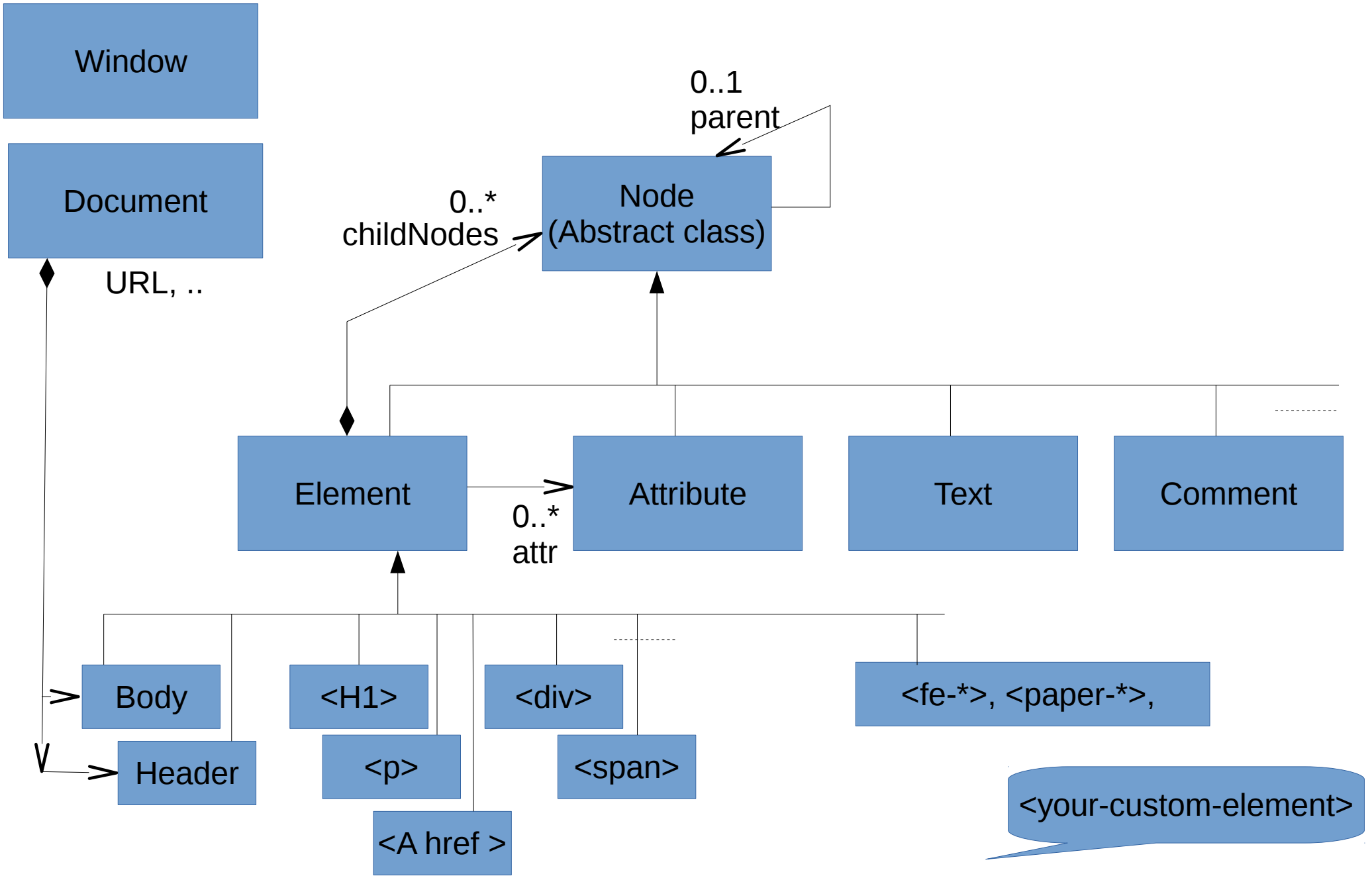
Common uses for JavaScript are image manipulation, form validation, and dynamic changes of content.

To select an HTML element, JavaScript very often use the document.getElementById(id) method.

Html / CSS / JavaScript Interactions



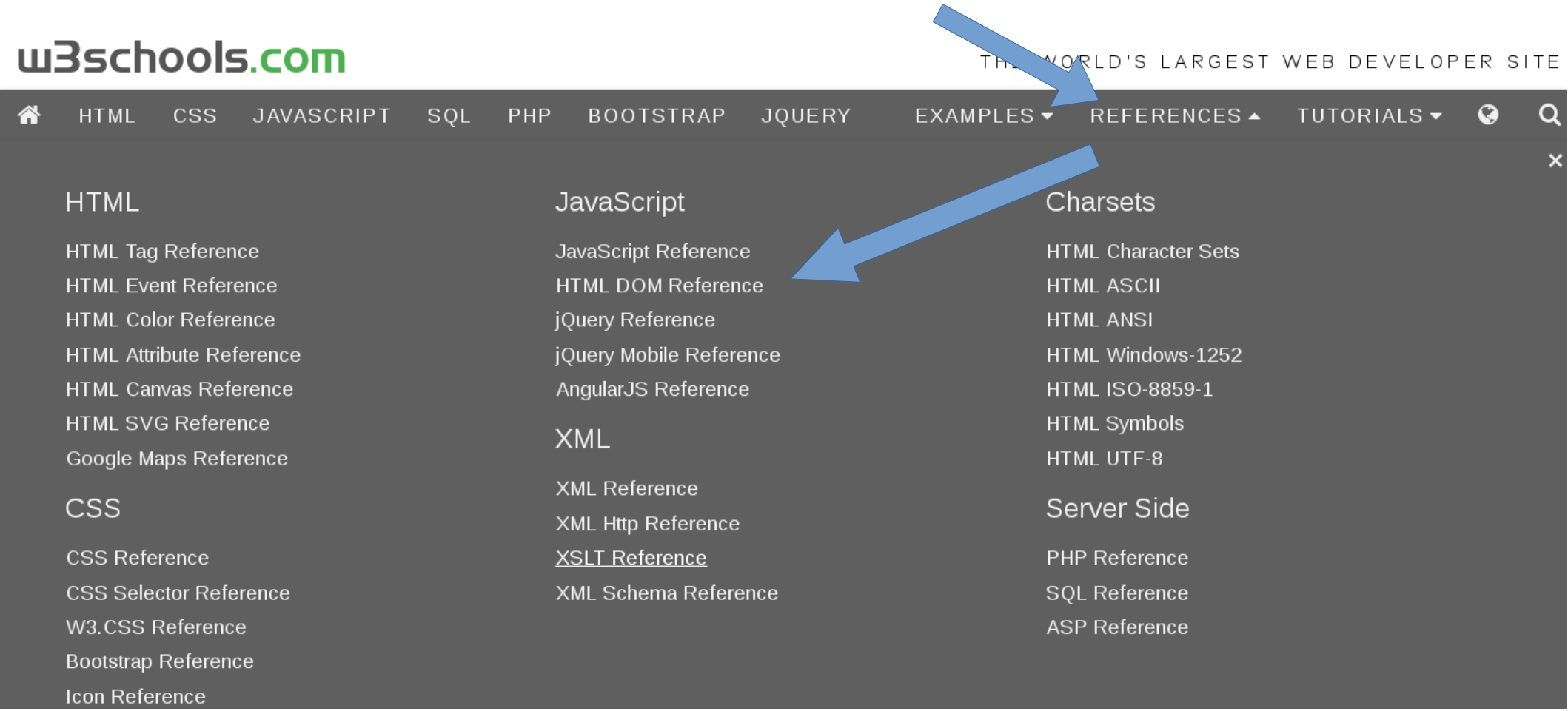
DOM = Document Object Model



DOM = Class Hierarchy

```
public abstract class Node {
   private Node parent;
   ..
   public Node getParent() { .. }
}

public abstract class Element extends Node
{
   private List<Node> childList;
   private List<Attribute> attributes;
   ..
}
```

DOM Reference



DOM Elements
Properties,Method,Events

HTML DOM
DOM Document
DOM Elements
DOM Attributes
DOM Events
DOM Style

HTML Objects
<a>
<abbr>
<address>
<area>
<article>
<aside>
<audio>
<b>
<base>
<bdo>
<blockquote>
<body>

<button>
<canvas>
<caption>

Properties and Methods

The following properties and methods can be used on all HTML elements:

Property / Method	Description
element.accessKey	Sets or returns the accesskey attribute of an element
element.addEventListener()	Attaches an event handler to the specified element
element.appendChild()	Adds a new child node, to an element, as the last child node
element.attributes	Returns a NamedNodeMap of an element's attributes
element.blur()	Removes focus from an element
element.childElementCount	Returns the number of child elements an element has
element.childNodes	Returns a collection of an element's child nodes (including text and comment nodes)
element.children	Returns a collection of an element's child element (excluding text and comment nodes)
element.classList	Returns the class name(s) of an element
element.className	Sets or returns the value of the class attribute of an element
element.click()	Simulates a mouse-click on an element
element.clientHeight	Returns the height of an element, including padding

DOM Elements Reference

HTML DOM

DOM Document
DOM Elements
DOM Attributes
DOM Events
DOM Style

HTML Objects

<a>
<abbr>
<address>
<area>
<article>
<aside>
<audio>
<b>
<base>
<bdo>
<blockquote>
<body>

<button>
<canvas>
<caption>
<cite>
<code>

HTML DOM Reference

The references describe the properties and methods of the HTML DOM, along with examples.

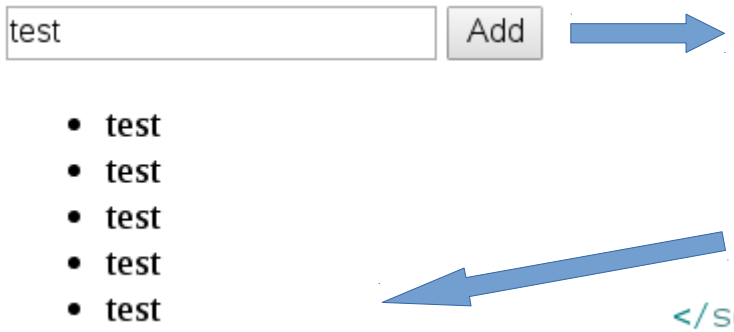
DOM Document	DOM Elements	DOM Attributes	DOM Events	DOM Style

HTML Element Objects Reference

The references describe the properties and methods of each HTML object, along with examples.

a	abbr	address	area	article	aside	audio	b	base	bdo	blockquote	body	br	button	canvas
caption	cite	code	col	colgroup	datalist	dd	del	details	dfn	dialog	div	dl	dt	em	embed
fieldset	figcaption	figure	footer	form	head	header	h1 - h6	hr	html	i	iframe	img	ins	input button
input checkbox	input color	input date	input datetime	input datetime-local	input email	input file	input hidden	input image						
input month	input number	input password	input radio	input range	input reset	input search	input submit	input text						
input time	input url	input week	kbd	keygen	label	legend	li	link	map	mark	menu	menuitem	meta	
meter	nav	object	ol	optgroup	option	output	p	param	pre	progress	q	s	samp	script	section
select	small	source	span	strong	style	sub	summary	sup	table	td	th	tr	textarea	time	title
track	u	ul	var	video										

Typical JS Code to Find & Edit Elts




```
<script>
    function onClickAddTodoItem() {
      let todoItemTextElt = document.getElementById('todoItemText');
      let text = todoItemTextElt.value;
      let newTodoItemElt = document.createElement("li");
      let newTodoItemTextElt = document.createTextNode(text);
      newTodoItemElt.appendChild(newTodoItemTextElt);
      let todoItemListElt = document.getElementById('todoItemList');
      todoItemListElt.appendChild(newTodoItemElt);
    }
</script>
<input id="todoItemText" type="text"></input>
<button type="button" onclick="onClickAddTodoItem()">Add</button>
<ul id="todoItemList"></ul>
```

Find:

Document
  .getElementById()

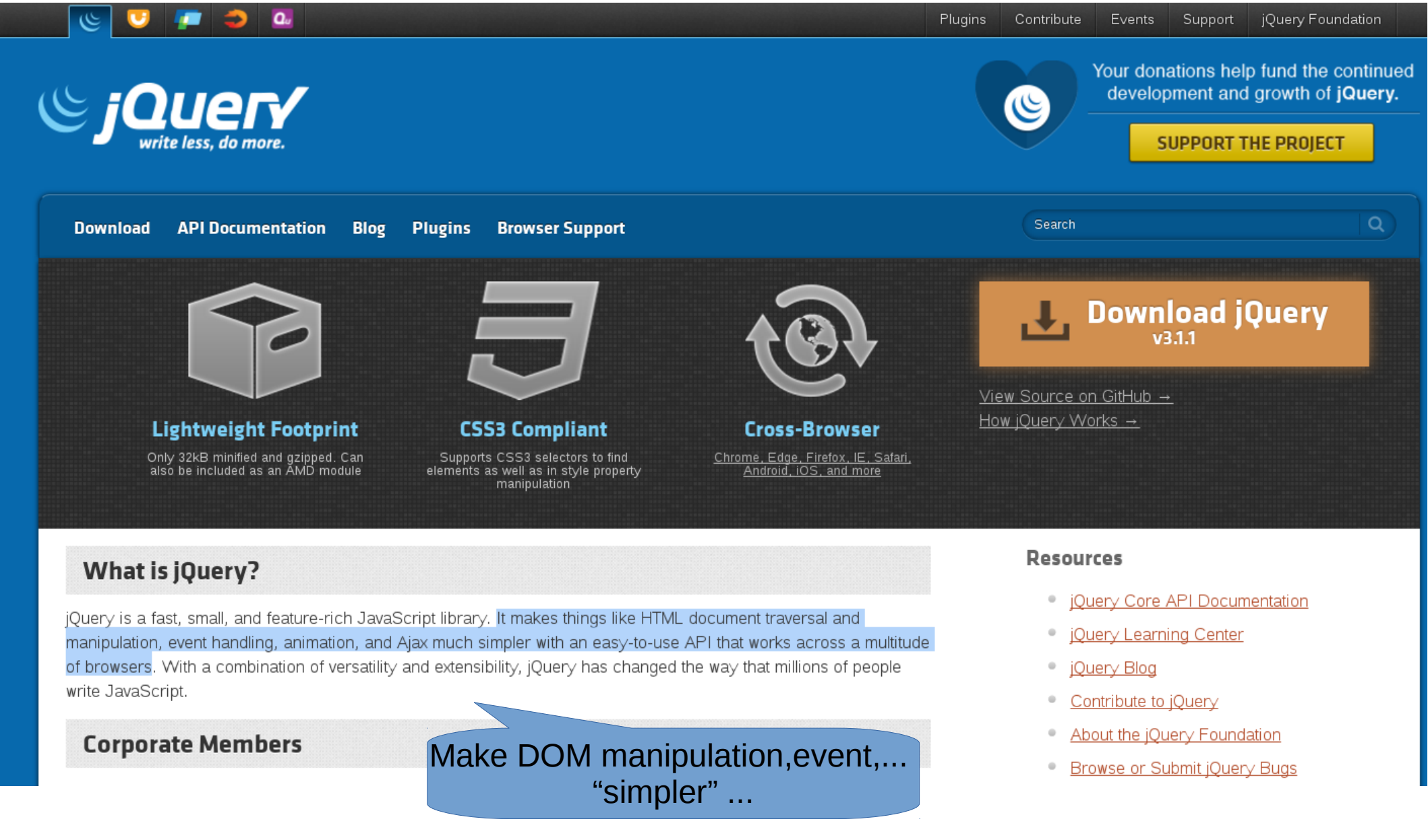
Create:

Document
  .createElement(“li”)
  .createTextNode()
...

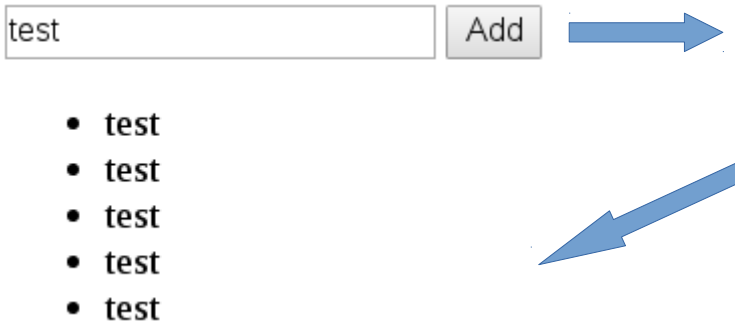
Update:

element.appendChild()
value = element.attribute
element.attribute = value
Element.innerHTML
...

JQuery for Portable/Concise DOM Manipulation?



Same TodoItem example using JQuery




```
<head>
  <script src="jquery.min.js"></script>
</head>
<body>
<script>
$(document).ready(function(){
    $("#todoItemAddButton").click(function(){
        let text = $("#todoItemText").val();
        $("#todoItemList").append("<li>" + text + "</li>");
    });
});
</script>
<input id="todoItemText" type="text"></input>
<button id="todoItemAddButton"type="button">Add</button>
<ul id="todoItemList"></ul>
</body>
```

Jquery $() : wrapper for Document.getElementBy..(..)

Code is smaller ??
by including “<script src=”jquery.min.js”></script>

Well ... If you use only 1 method of Jquery...

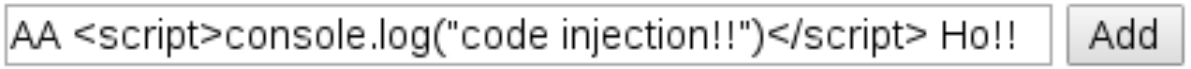
You could have use “jqueryLight”

or even

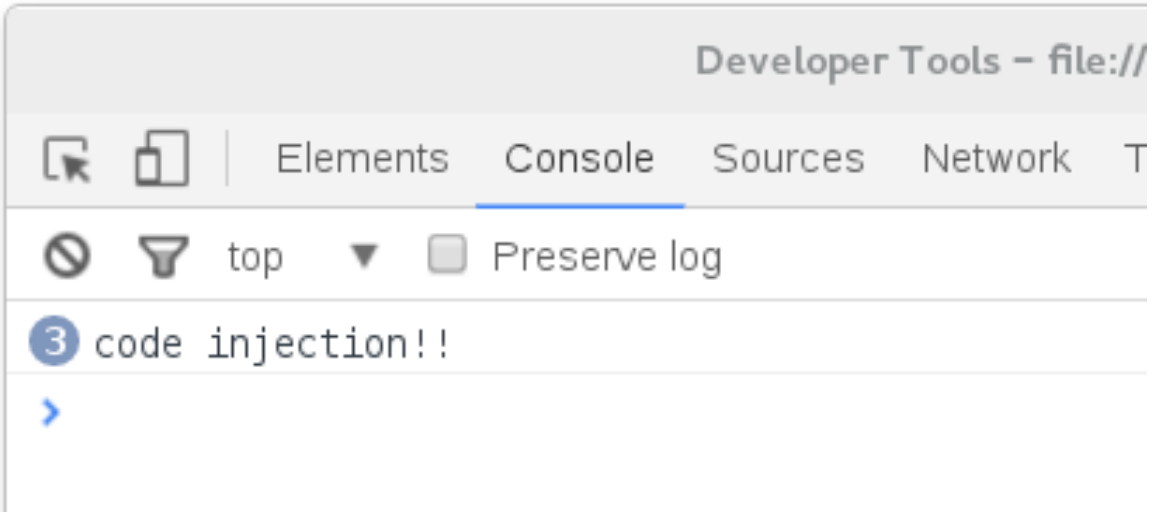
```
function my$(id) {
   return Document.getElementById(id);
}
```

Jquery Risk… Script Code Injection!

Example in previous example :
append(“
” + text + “”)
… text can contain html script!!



- AA Ho!!
- AA Ho!!
- AA Ho!!



Jquery vs DOM

Naive retrial after google it … same pb !!

```
$('<li />', {html: text}).appendTo('#todoItemList');
```

This one works ...

```
$("#todoItemList").append($('<li />').append(document.createTextNode(text)));
```

AA <script>console.log("code injection!!")</script> Ho!! Add

- **test**
- **test**
- **AA <script>console.log("code injection!!")</script> Ho!!**
- **AA <script>console.log("code injection!!")</script> Ho!!**

Compare with Explicit DOM + findById

```
function findById(id) { return document.getElementById(id); }
function onClickAddTodoItem2() {
  let text = findById('todoItemText').value;
  let newTodoItemElt = document.createElement("li");
  newTodoItemElt.appendChild(document.createTextNode(text));
  findById('todoItemList').appendChild(newTodoItemElt);
}
```

Jquery Bashing in 2016 ...

$() code not simple ! … not “typed”

JQuery is a very low-level framework
… for writing mostly imperative code
Or applying plugins

No easy bi-direction Binding
Better use Richer Framework, like AngularJS

Browser Portability no more an issue in 2016?

ANGULARJS
Home
Learn
Develop
Discuss
Search
ANGULARJS
by Google
HTML enhanced for web apps!
Download AngularJS 1
(1.6.0-rc.2 / 1.5.9 / 1.2.32)
Try the new Angular 2
View on GitHub
Design Docs & Notes
Follow +AngularJS on
Follow @angularjs
187K followers
Tweet
Learn Angular in your browser for free!

Same TodoItem example using AngularJS-1.x

```
<!DOCTYPE html>
<html>
<head>
  <script src="angular.min.js"></script>
<script>
angular.module("myApp", []).controller("MyController", function() {
   var self = this;
   self.todoText = '';
   self.todoItems = [];
   self.onClickAddTodoItem = function() {
     self.todoItems.push(self.todoText);
   };
});
</script>
</head>
<body ng-app="myApp">
<div ng-controller="MyController as ctrl">
    <input ng-model="ctrl.todoText" type="text" size="40"></input>
    <button ng-click="ctrl.onClickAddTodoItem()" type="button">Add</button>
    <ul>
        <li ng-repeat="item in ctrl.todoItems track by $index">{{item}}</li>
    </ul>
</div>
</body>
</html>
```

DOM vs Jquery vs AngularJS

- AngularJS code ~ 25 lines / jquery ~ 21 lines
- AngularJS code much more readable

Imperative code

Direct DOM manipulation

NO Binding, Events

events history required

Declarative code

MVC : Model-View-Controller

Bi-Directionnal Binding

(no events required, only Model)

Much more to say on Angular MVC Binding...

- very handy writing quick web app
- not very efficient having 1000 objects in scope
- ...digest() loop is horrible

- Angular 2.x is complete rewrite of 1.x
- ...still not very clean code (digest zone?)
  Model should be controlled.. and fire chg Event

- ReactJs, Aurelia, ..

  … JS Word = “1 fwk a day, the doctor away”?

Conclusion

- Only a “very short” introduction to Html-Js-Dom
- I Hope you feel
  - “I Did not know all this before”
  - “I am happy I have learned something”
  - “I want to learn more”

This document:
http://arnaud-nauwynck.github.io/docs/Intro-Html-JS-DOM.pdf